

churn 모델

churn 모델

<4/30 8:07 업데이트>

TRAIN 0.8062

VAILD 0.8013

앙상블 : 절사값0.35, VOTING,SVM 의미X

챔피언	이름	알고리...	ROC 아...	정확도	평균제...	누적향...	누적반...	점수
★	앙상블	앙상블	0.8013	0.9131	0.0708	3.9179	39.1793	0.8013
	GB (1)	그래디언트 부스팅	0.8007	0.9131	0.0709	3.8617	38.6172	0.8007
	GB (2)	그래디언트 부스팅	0.8001	0.9134	0.0708	3.8898	38.8983	0.8001
	GB (4)	그래디언트 부스팅	0.8001	0.9131	0.0709	3.9348	39.3479	0.8001
	GB (3)	그래디언트 부스팅	0.8013	0.9131	0.0708	3.9882	39.8820	0.8013
	GB	그래디언트 부스팅	0.8010	0.9128	0.0708	3.9067	39.0669	0.8010

적합통계량

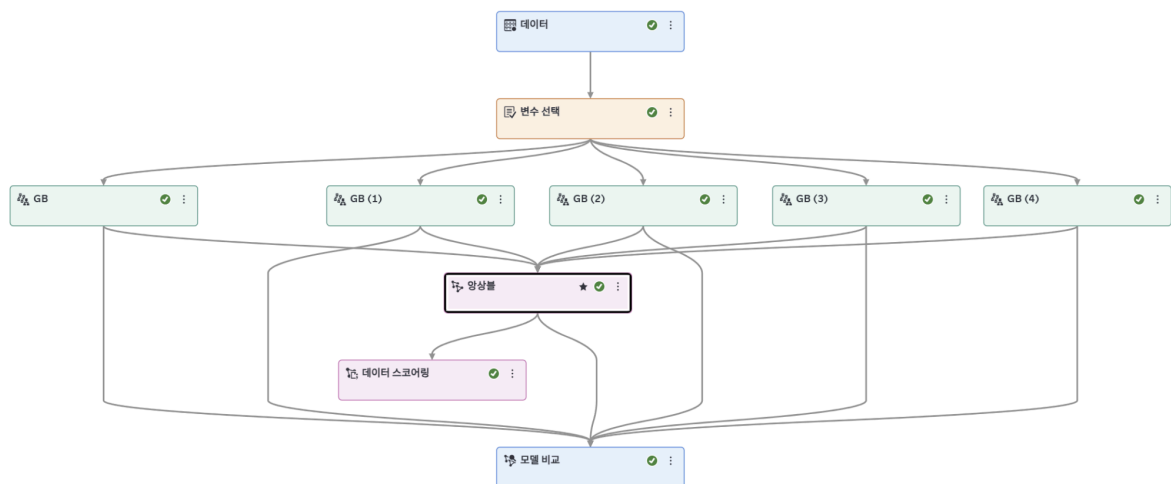
타겟 이름	데이터 역할	분할 표시자	출력형식이 지정된 분할
target_churn	TRAIN	1	1
target_churn	VALIDATE	0	0

관측값 수	평균제곱오차	ASE의 분모	제곱근평균제곱오차
42,000	0.0691	42,000	0.2629
18,000	0.0708	18,000	0.2660

오분류 비율	다중 Class 로그 손실	KS(Youden)	ROC 아래면적
0.0840	0.2483	0.4902	0.8062
0.0869	0.2522	0.4991	0.8013

Gini 계수	감마	Tau	KS 절사값
0.6123	0.6433	0.1091	0.0800
0.6026	0.6332	0.1073	0.1100

사용자 지정 절사값에서의 KS	KS 절사값에서의 오분류 비율(이벤트)	오분류 비율(이벤트)
0.1874	0.2124	0.0840
0.1714	0.1867	0.0869



<4/28 8:17 업데이트>

5-



fold CV 진행결과 : 변수선택 -> **GB 0.80064**

챔피언	이름	알고리즘...	ROC 아...	정확도	평균제곱...	누적합...	누적반응...
★	GB (3)	그래디언트 부스팅	0.8013	0.9131	0.0708	3.9882	39.8820
	GB (1)	그래디언트 부스팅	0.8007	0.9131	0.0709	3.8617	38.6172
	GB (2)	그래디언트 부스팅	0.8001	0.9134	0.0708	3.8898	38.8983
	GB (4)	그래디언트 부스팅	0.8001	0.9131	0.0709	3.9348	39.3479
	GB	그래디언트 부스팅	0.8010	0.9128	0.0708	3.9067	39.0669

1. 변수 역할과 레벨 설정

target_churn: Target / Binary

customer_id: ID / Nominal

target_ltv, target_ltv_log: Rejected / Interval

gender, is_married: Input / Binary

income_group, prefer_category, region_code: Input / Nominal

나머지 거래금액, 거래건수, 잔고, 비율, 점수, 기간, 일수, 개수 변수: Input / Interval

☐	변수 이름 ↑	역할	레벨
☐	active_months_6m	Input	Interval
☐	age	Input	Interval
☐	amt_month_slope	Input	Interval
☐	card_cash_service_amt_0	Input	Binary
☐	card_cash_service_amt_log	Input	Interval
☐	card_loan_amt_log	Input	Interval
☐	category_nunique_6m	Input	Interval
☐	credit_score	Input	Interval
☐	customer_id	ID	Nominal
☐	days_since_last_txn	Input	Interval
☐	fin_asset_trend_score	Input	Interval
☐	fin_overdue_days	Input	Interval
☐	gender	Input	Binary
☐	income_group	Input	Nominal
☐	installment_ratio_6m	Input	Interval
☐	is_married	Input	Binary
☐	loan_to_deposit_cut	Input	Interval
☐	net_balance	Input	Interval
☐	num_active_cards	Input	Interval
☐	online_ratio_6m	Input	Interval
☐	prefer_category	Input	Nominal
☐	recent_amt_share_1m	Input	Interval
☐	recent_amt_share_3m	Input	Interval
☐	region_code	Input	Nominal
☐	target_churn	Target	Binary
☐	target_ltv	Rejected	Interval
☐	target_ltv_log	Rejected	Interval
☐	tenure_days	Input	Interval
☐	total_deposit_balance_log	Input	Interval
☐	total_loan_balance_log	Input	Interval
☐	txn_amt_mean_6m	Input	Interval
☐	txn_amt_std_6m	Input	Interval
☐	txn_amt_sum_1m	Input	Interval
☐	txn_amt_sum_3m	Input	Interval
☐	txn_amt_sum_6m	Input	Interval
☐	txn_cnt_1m	Input	Interval
☐	txn_cnt_3m	Input	Interval
☐	txn_cnt_6m	Input	Interval

1.2 프로젝트 설정

- 데이터 분할 : 프로젝트 설정 - 데이터 분할 70: 30 - 방법:층화

층화 분할: `target_churn`이 0/1이잖아. 예를 들어 전체 데이터가 이런 비율이라고 해보자: 유지(0): 90% 이탈(1): 10% / 그런데 아무 생각 없이 랜덤 분할하면 Validation 쪽에 이탈 고객이 너무 적거나 많아질 수 있어. 그래서 `target_churn`의 0/1 비율을 Train과 Validation에 비슷하게 맞춰서 나누는 것이 층화 분할이야. 예:전체: 0이 90%, 1이 10% / Train도: 0이 90%, 1이 10% / Validation도: 0이 90%, 1이 10% 층화 분할을 안 하면 어떤 쪽에는 이탈 고객이 너무 적고 어떤 쪽에는 너무 많아서 성능 평가가 흔들릴 수 있어. 그래서 `target_churn` 기준으로 stratified partition 하라고 한 거야.

프로젝트 설정

데이터 분할

이벤트 기반 표본추출

노드 구성

규칙

출력 라이브러리

로깅

컴퓨팅 컨텍스트

데이터 분할

☒ 분할변수 생성

참고: 이 설정은 분할변수가 데이터 내에 설정되지 않을 때만 활성화됩니다. 미리 정의된 분할변수가 있는 데이터 소스를 사용하거나 분할변수를 수동으로 선택하면 설정이 재정의됩니다.

방법:
중화

학습: *
70 70.00%

검증: *
30 30.00%

테스트: *
0 0.00%

저장 취소

● 규칙

Class 모델 선택 통계량: 곡선아래면적(C 통계량)

이유: SAS에서 C 통계량은 이진분류에서 사실상 ROC-AUC와 같은 계열이라서, 대회 평가 기준과 가장 직접적으로 맞아.

프로젝트 설정

데이터 분할

이벤트 기반 표본추출

노드 구성

규칙

출력 라이브러리

로깅

컴퓨팅 컨텍스트

규칙

모델 비교

Class 모델 선택 통계량:
곡선아래면적(C 통계량)

Interval 모델 선택 통계량:
평균재검오차

모델 선택 분할:
기본

기본 선택은 사용 가능 여부에 따라 테스트, 검증, 학습 순입니다.

모델 선택 깊이:
10

ROC 기반 절사값:
0.50

모델

저장 취소

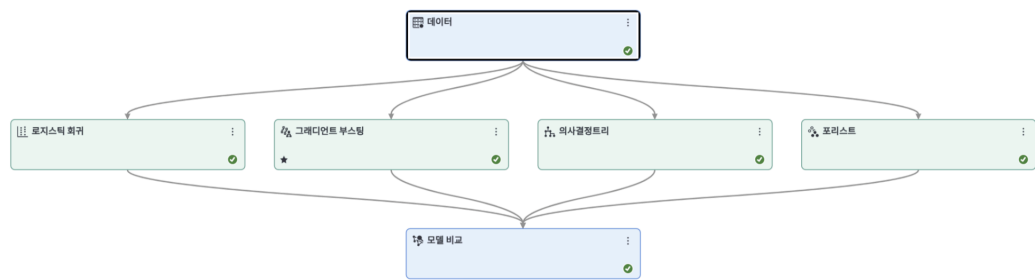
2. 알고리즘 탐색

2.1 챔피언 모델 선정 - 그래디언트부스팅

대회 평가 기준인 ROC-AUC를 1순위 지표로 적용하여 초기 4개 모델의 성능을 비교 분석했습니다.

● 핵심 지표: VALIDATE ROC 아래면적 (AUC)

- 보조 지표: Train vs Validate AUC 차이 (과적합 진단)



모델명	VALIDATE AUC (성능 순위) (test)	TRAIN AUC (train)	AUC 차이 (과적합 진단) (test-train)	주요 해석 및 결론
그래디언트 부스팅	0.7929 (1위)	0.8306	약 0.038 (양호)	챔피언 모델 선정. 성능이 가장 우수하며 과적합 신호가 적어 가장 균형 잡힌 모델.
랜덤 포리스트	0.7850 (2위)	0.8960	약 0.111 (주의)	훈련 성능이 매우 높으나 검증에서 크게 하락. 과적합 신호가 강함. 튜닝을 통한 완화 필요.
의사결정트리	0.7604 (3위)	0.7558	차이 작음 (안정적)	성능은 낮으나 과적합은 없음. 기본 모델로 활용 가능.
로지스틱 회귀	0.7294 (4위)	0.7203	차이 작음 (안정적)	성능이 가장 낮음. 선형 모델의 한계.

- 초기 성능 해석:
- 챔피언 모델 선정 (AUC 기준): 그래디언트 부스팅 모델이 검증 AUC 0.7929로 가장 우수하며, 보조 지표인 KS도 0.4962로 가장 높아 제출용 챔피언 모델로 우선 선정합니다.
 - 과적합 진단: 랜덤 포리스트는 Train AUC가 0.8960으로 높으나 Validate AUC와의 차이가 0.111로 커서 과적합 신호가 강하게 나타났습니다. 그래디언트 부스팅은 상대적으로 그 차이가 0.038로 양호한 편입니다.
 - Accuracy/F1 해석: 데이터 불균형으로 인해 Accuracy만으로는 모델 성능을 정확히 판단하기 어려우며, 현재 F1-Score는 절사값(Cutoff) 0.5 기준으로 낮게 측정되었으나, AUC 및 KS 성능이 우수하므로 확률 정렬 능력은 높다고 판단합니다.

-그래디언트 부스팅 하이퍼파라미터 튜닝 후 성능이 정체될 시 파생변수를 추가하는 순으로 작업 진행(파생변수는 시간이 더 많이 들고, 잘못 만들면 오히려 잡음이 늘어날 수도 있음)

조기중지 옵션 on & 로그손실

- 더 학습해도 성능이 안 좋아지면 일찍 멈추는 장치. GB는 트리를 계속 추가하면서 학습하니까, 너무 오래 가면 훈련 데이터에 과하게 맞춰질 수 있어. 즉, **성능 + 시간 + 과적합 방지** 때문에 켜는 거야.

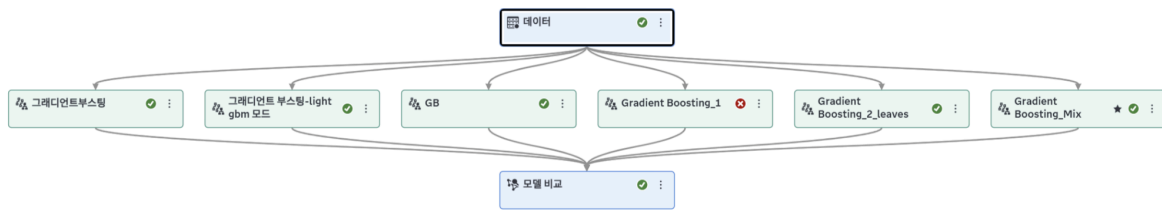
오분류 비율의 문제 오분류 비율은 보통 **0.5 절사값** 기준이야. 즉 예측확률이 0.49, 0.30, 0.10 이 셋은 전부 0으로 잘려서 같은 취급이 될 수 있어.

근데 churn 대회는 확률 순서가 중요하잖아. AUC는 “누가 더 위험한 고객인지”를 얼마나 잘 정렬했는지를 봐. 로그손실은 예측확률 자체를 반영해. 실제 이탈 고객에게 0.49를 줬는지 0.10을 줬는지 차이를 반영해. 오분류율보다 확률 예측 품질을 더 잘 반영해.

데이터 탐색에서 변수 중요도 및 상관관계 파악

gb: 파라미터 지정, 절사값 변경(이벤트 분류 창에서 다양한 겨우 볼 수 있음)
모델비교:

2.2 GB 모델 선정 - Gradient Boosting_Mix 0.7966



모델 비교

챔피언	이름	알고리즘 이름	ROC 아래면적
true	Gradient Boosting_Mix	그래디언트 부스팅	0.7966
false	그래디언트부스팅	그래디언트 부스팅	0.7929
false	그래디언트 부스팅-light gbm 모드	LightGBM	0.7934
false	GB	그래디언트 부스팅	0.7933
false	Gradient Boosting_2_leaves	그래디언트 부스팅	0.7946

2.3 절사값 변경

챔피언	이름	알고리...	ROC 아...	정확도	평균제...
★	Gradient Boosting_Mix (1)	그래디언트 부스팅	0.7966	0.9129	0.0721
	Gradient Boosting_Mix 절사값0.3	그래디언트 부스팅	0.7966	0.9129	0.0721
	Gradient Boosting_Mix 변수 관리	그래디언트 부스팅	0.7956	0.9138	0.0712

1. 0.5
2. 0.3

3. 0.5+하위변수 `is_married`(결혼 여부), `gender`(성별), `region_code`(거주 지역) 3개 제외.
- 4.

2.4 하이퍼 파라미터 튜닝

2.4 하이퍼 파라미터 튜닝 비교 및 추천 (그래디언트 부스팅)

파라미터	1안. 균형형 (1순위 추천)	2안. 보수형 (과적합 완화)	3안. 공격형 (성능 극대화)
목표 해석	과적합과 성능의 균형. 가장 먼저 시도	과적합 최소화, 안정적 결과 유도	패턴 적극 탐색, Recall/AUC 향상 시도
트리 수	150	100	250
학습률	0.05	0.03	0.03
부분 표본 비율	0.8	0.8	0.6
L1 정규화	0.1	0.3	0.05
L2 정규화	0.3	0.8	공란
최대 가지 수	3	2	4
최대 깊이	4	3	5
최소 리프 크기	20	30	10
타깃 분포	이항 또는 churn용 분류 옵션	공란	공란
시드	12345	12345	12345
결측값 처리	탐색에 사용	탐색에 사용	탐색에 사용
최소 결측값	1	공란	공란
Interval 구간 수	50	50	50

-----> 파생변수 & 데이터 마이닝 사전처리 로 모델 성능 향상시키는 방향으로 가야 함.

2.2 데이터 마이닝 사전처리

변수관리, 변수선택, 변환 etc..

모델 비교					
챔피언	이름	알고리...	ROC 아...	정확도	평균제..
★	Gradient Boosting_Mix	그래디언트 부스팅	0.7996	0.9137	0.07
	포리스트	포리스트	0.7924	0.9138	0.07
	의사결정트리	의사결정트리	0.7613	0.9127	0.07
	앙상블	앙상블	0.7957	0.9129	0.07

결과: Variable Selction2 + GB MIX 0.7996

3. 파이썬 연동

XGB(파이썬, 5-fold CV) 0.796681
Variable Selction2+XGB(파이썬, 5-fold CV) 0.796681
GB(파이썬, 5-fold CV) 0.748851
Variable Selction2 + GB(파이썬, 5-fold CV) 0.805787

3.1 파이썬 모델링 결과 분석 (AUC 기준)

모델명	AUC (5-fold CV)
XGB (파이썬)	0.796681 ▾
Var.Sel. 2 + XGB	0.796681 ▾
GB (파이썬)	0.748851 ▾
Var.Sel. 2 + GB	0.805787 ▾

최종 결론:

- **Variable Selection 2 + GB (파이썬)** 모델이 **AUC 0.805787**로 현재까지 가장 우수한 성능을 보였습니다. 해당 모델을 최종 제출 모델로 확정하고 추가적인 미세 튜닝을 진행할 예정입니다.
- 변수 선택(Variable Selection 2)은 특히 Gradient Boosting 계열 모델의 성능을 크게 끌어올리는 데 효과적이었습니다.

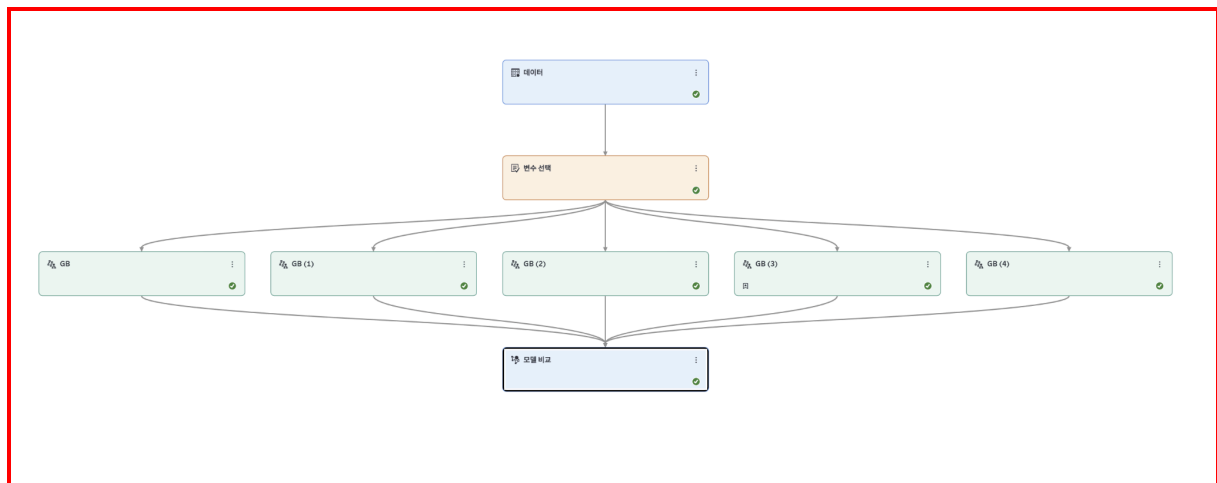
Todo

variable selection2 미세 조정하기! -> sas viya 도움말.

문제. cv 진행 전 결과임.

<4/28 8:17 업데이트>

5-fold CV 진행결과 : 변수선택 -> **GB 0.80064**



챔피언	이름	알고리즘...	ROC 아...	정확도	평균제곱...	누적항...	누적반응...
★	GB (3)	그래디언트 부스팅	0.8013	0.9131	0.0708	3.9882	39.8820
	GB (1)	그래디언트 부스팅	0.8007	0.9131	0.0709	3.8617	38.6172
	GB (2)	그래디언트 부스팅	0.8001	0.9134	0.0708	3.8898	38.8983
	GB (4)	그래디언트 부스팅	0.8001	0.9131	0.0709	3.9348	39.3479
	GB	그래디언트 부스팅	0.8010	0.9128	0.0708	3.9067	39.0669

하이퍼

실험	트리 수	학습률	부분 표본 비율	최대 깊이	최소 리프 크기	L1	L2
현재 기준	70	0.1	0.5	4	5	0	1
A 안정형	100	0.07	0.5	4	5	0	1
B 정밀형	120	0.05	0.5	4	5	0	1
C 복잡도↑	100	0.07	0.5	5	5	0	1
D 과적합 완화	100	0.07	0.6	4	10	0	1
E 규제 강화	120	0.05	0.5	4	5	0	3

□ABC갓다버려

GB계열비교-GB선정

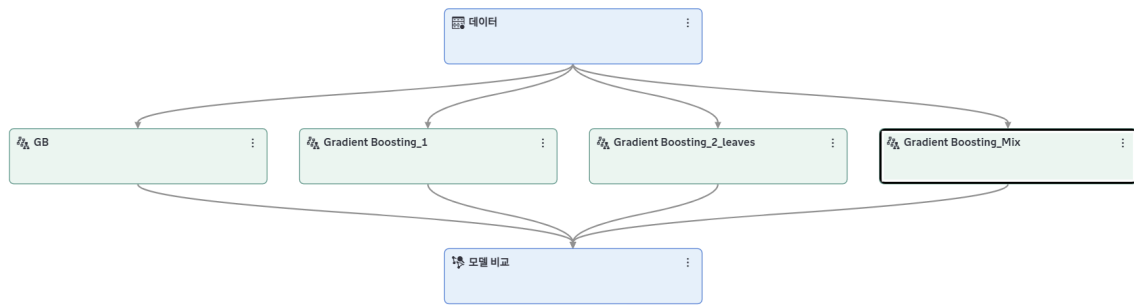
fix : 5-fold 시드 1,2,3,4,5

순 위	모델명	평균 ROC-AUC (Mean)	표준편차 (SD)	비고
1	2. GB	0.7969	0.0009	🏆 성능 1위, 안정성 1위
2	4. Gradient Boosting 2 leaves	0.7960	0.0015	훌륭한 대안 (단순한 트리 구조)
3	5. Gradient Boosting Mix	0.7959	0.0022	성능 우수
4	1. 그래디언트부스팅	0.7954	0.0029	준수한 베이스라인
5	6. xgb (XGBoost)	0.7936	0.0096	성능 무난, Fold 간 편차 큼
6	3. Gradient Boosting_1	0.7819	0.0010	성능 다소 아쉬움

1. 최종 선택의 승자: 2번 "GB" 모델

- 현재 테스트하신 모델 중 2번 GB 모델이 압도적인 1픽(Pick)입니다.
- 평가 지표인 AUC 평균이 0.79692로 가장 높을 뿐만 아니라, 표준편차(SD)가 0.0009라는 것은 5번의 Fold 테스트 내내 성능이 흔들리지 않고 매우 안정적이었다는 것을 의미합니다. 즉, Test 데이터(Public/Private 리더보드)에서도 과적합 없이 비슷한 성능을 낼 확률이 아주 높습니다

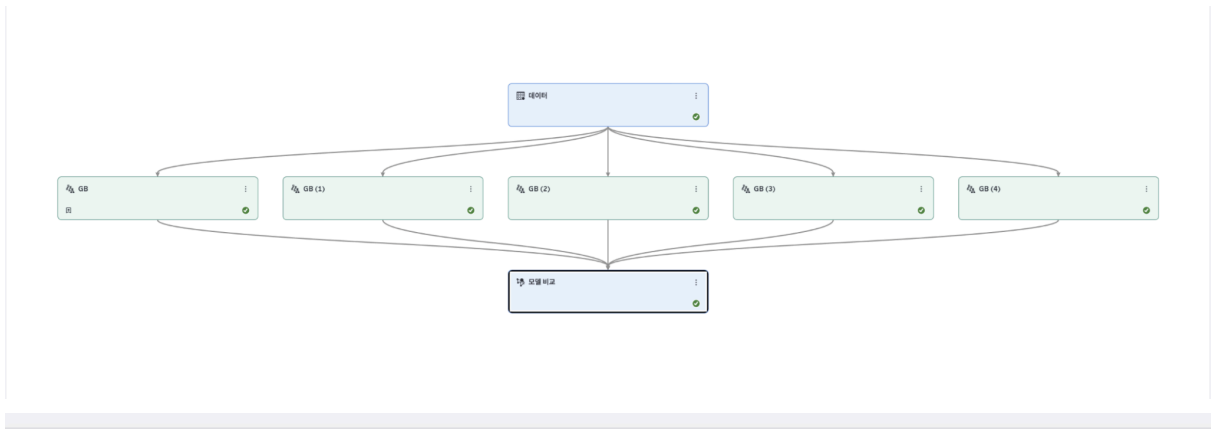
1.그래디언트부스팅



챔피언	이름	알고리...	ROC 아...	
★	그래디언트 부스팅 (3)	그래디언트 부스팅	0.7977	
	그래디언트 부스팅	그래디언트 부스팅	0.7905	
	그래디언트 부스팅 (1)	그래디언트 부스팅	0.7971	
	그래디언트 부스팅 (2)	그래디언트 부스팅	0.7958	
	그래디언트 부스팅 (4)	그래디언트 부스팅	0.7963	

mean: 0.79548
sd: 약 0.0029

[2.GB](#)



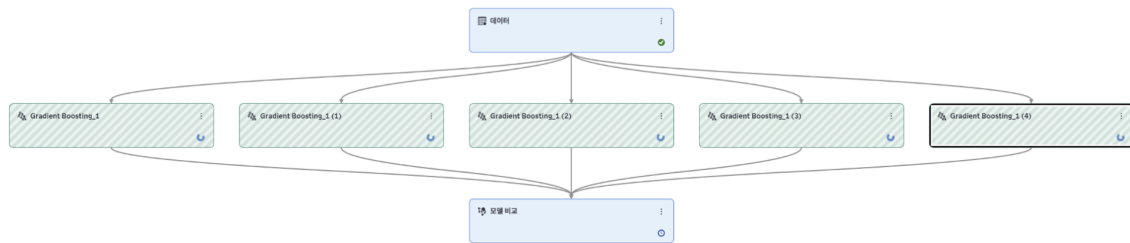
모델 비교

챔피언	이름	알고리즘 ...	ROC 아...	정확도	평균제곱...
★	GB	그래디언트 부스팅	0.7977	0.9129	0.0712
	GB (1)	그래디언트 부스팅	0.7971	0.9131	0.0712
	GB (2)	그래디언트 부스팅	0.7958	0.9134	0.0712
	GB (3)	그래디언트 부스팅	0.7977	0.9132	0.0710
	GB (4)	그래디언트 부	0.7963	0.9130	0.0713

mean: 0.79692

sd: 0.0009

3. Gradient Boosting_1



챔피언	이름	알고리즘...	ROC 아...	정확도	평균제곱...	누적합...	누적반응...
★	Gradient Boosting_1	그래디언트 부스팅	0.7836	0.9127	0.0733	3.7999	37.9989
	Gradient Boosting_1 (1)	그래디언트 부스팅	0.7816	0.9121	0.0742	3.6987	36.9871
	Gradient Boosting_1 (2)	그래디언트 부스팅	0.7817	0.9116	0.0738	3.7493	37.4930
	Gradient Boosting_1 (3)	그래디언트 부스팅	0.7818	0.9118	0.0734	3.8280	38.2799
	Gradient Boosting_1 (4)	그래디언트 부스팅	0.7808	0.9114	0.0735	3.7774	37.7740

ROC-AUC 값

0.7836

0.7816

0.7817

0.7818

0.7808

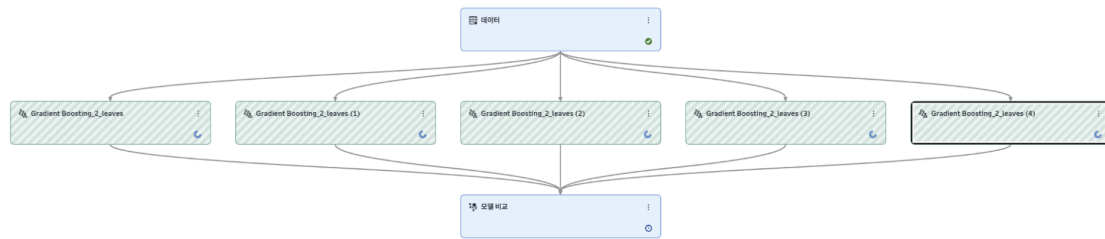
평균 (Mean)

0.7819

표준편차 (SD)

0.0010

4.Gradient Boosting 2 leaves



모델 비교

챔피언	이름	알고리즘...	ROC 아...	정확도	평균제곱...	누적항...	누적반응...	절사값	데이터 ...
★	Gradient Boosting_2_leaves (3)	그래디언트 부스팅	0.7985	0.9135	0.0710	3.9179	39.1793	0.5000	VALIDAT
	Gradient Boosting_2_leaves	그래디언트 부스팅	0.7958	0.9127	0.0713	3.7886	37.8865	0.5000	VALIDAT
	Gradient Boosting_2_leaves (1)	그래디언트 부스팅	0.7947	0.9125	0.0715	3.8730	38.7296	0.5000	VALIDAT
	Gradient Boosting_2_leaves (2)	그래디언트 부스팅	0.7949	0.9130	0.0712	3.8449	38.4486	0.5000	VALIDAT
	Gradient Boosting_2_leaves (4)	그래디언트 부스팅	0.7963	0.9129	0.0713	3.7999	37.9989	0.5000	VALIDAT

ROC-AUC 값

0.7985

0.7958

0.7947

0.7949

0.7963

평균 (Mean)

0.7960

표준편차 (SD)

0.0015

5. Gradient Boosting Mix



챔피언	이름	알고리즘...	ROC 아...	정확도	평균제공...	누적항...	누적반응...	절사값	데이터 ...	
★	Gradient Boosting_Mix (3)	그래디언트 부스팅	0.7985	0.9126	0.0713	3.9011	39.0107	0.5000	VALIDATE	
	Gradient Boosting_Mix	그래디언트 부스팅	0.7966	0.9131	0.0716	3.8055	38.0551	0.5000	VALIDATE	
	Gradient Boosting_Mix (1)	그래디언트 부스팅	0.7950	0.9121	0.0715	3.8786	38.7858	0.5000	VALIDATE	
	Gradient Boosting_Mix (2)	그래디언트 부스팅	0.7967	0.9129	0.0715	3.8055	38.0551	0.5000	VALIDATE	
	Gradient Boosting_Mix (4)	그래디언트 부스팅	0.7925	0.9123	0.0716	3.8392	38.3924	0.5000	VALIDATE	

ROC-AUC 값

0.7985

0.7966

0.7950

0.7967

0.7925

평균 (Mean)

0.7959

표준편차 (SD)

0.0022

6. xgb

XGBoost 5-Fold CV 결과

AUC 평균: **0.793594**

AUC 표준편차: 0.0096

```
# =====
# SAS Viya Python Node용
# 최종 전처리 데이터 기반 XGBoost 5-Fold CV + Scoring
# 기준 지표: ROC-AUC
# =====

import pandas as pd
import numpy as np

from xgboost import XGBClassifier
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import roc_auc_score, accuracy_score, f1_score, log_loss,
confusion_matrix

# =====
# 1. SAS Viya 입력 데이터
# =====

df = dm_inputdf.copy()

print("==== df shape ====")
print(df.shape)

print("==== columns ====")
print(df.columns.tolist())

# =====
# 2. 기본 설정
# =====

target_col = "target_churn"

# Churn 예측에서 절대 사용하면 안 되는 누수 변수
exclude_cols = [
    "customer_id",
    "target_churn",
    "target_itv",
    "target_itv_log",
    "_dmIndex_",
```

```

    "_PartInd_"
]

# 최종 전처리 데이터에 남아 있을 수 있는 범주형 변수
cat_cols = [
    "gender",
    "region_code",
    "prefer_category",
    "income_group",
    "is_married",
    "overdue_flag"
]

# =====
# 3. 전처리 함수
# =====

def preprocess_data(input_df, train_columns=None):
    actual_exclude = [c for c in exclude_cols if c in input_df.columns]

    X = input_df.drop(columns=actual_exclude, errors="ignore")
    y = input_df[target_col].astype(int) if target_col in input_df.columns else None

    actual_cat_cols = [c for c in cat_cols if c in X.columns]

    X_dummy = pd.get_dummies(
        X,
        columns=actual_cat_cols,
        drop_first=True
    )

    if train_columns is not None:
        X_dummy = X_dummy.reindex(columns=train_columns, fill_value=0)

    X_dummy = X_dummy.fillna(0)

    return X_dummy, y

# =====
# 4. CV용 데이터 준비
# =====

# SAS 파티션이 있으면 Train 부분만 CV에 사용
if "_PartInd_" in df.columns:
    train_df = df[df["_PartInd_"].astype(str) == "1"].copy()
else:

```

```

train_df = df.copy()

X_train, y_train = preprocess_data(train_df)
train_cols = X_train.columns

print("==== X_train shape ====")
print(X_train.shape)

print("==== y_train distribution ====")
print(y_train.value_counts(dropna=False))

# =====
# 5. 누수 변수 확인
# =====

leakage_keywords = [
    "target",
    "ltv",
    "customer_id",
    "_dmIndex_",
    "_PartInd_"
]

suspicious_cols = [
    col for col in X_train.columns
    if any(key.lower() in col.lower() for key in leakage_keywords)
]

print("==== Leakage Check ====")
print("누수 의심 컬럼:", suspicious_cols)

if len(suspicious_cols) > 0:
    raise ValueError("누수 의심 컬럼이 X_train에 남아 있습니다. exclude_cols를 확인하세요.")

# =====
# 6. 클래스 불균형 보정
# =====

neg = (y_train == 0).sum()
pos = (y_train == 1).sum()

scale_pos_weight = neg / pos

print("scale_pos_weight:", scale_pos_weight)

```

```
# =====  
# 7. XGBoost 모델 함수  
# =====
```

```
def make_xgb_model(random_state=42):  
    return XGBClassifier(  
        n_estimators=300,  
        max_depth=3,  
        learning_rate=0.05,  
        subsample=0.8,  
        colsample_bytree=0.8,  
        min_child_weight=5,  
        gamma=0.3,  
        reg_lambda=5.0,  
        reg_alpha=0.5,  
        scale_pos_weight=scale_pos_weight,  
        objective="binary:logistic",  
        eval_metric="auc",  
        random_state=random_state,  
        n_jobs=-1  
    )
```

```
# =====  
# 8. Stratified 5-Fold CV  
# =====
```

```
skf = StratifiedKFold(  
    n_splits=5,  
    shuffle=True,  
    random_state=42  
)
```

```
cv_results = []
```

```
for fold, (tr_idx, va_idx) in enumerate(skf.split(X_train, y_train), start=1):
```

```
    X_tr = X_train.iloc[tr_idx]  
    X_va = X_train.iloc[va_idx]  
    y_tr = y_train.iloc[tr_idx]  
    y_va = y_train.iloc[va_idx]
```

```
    model = make_xgb_model(random_state=42)  
    model.fit(X_tr, y_tr)
```

```
    prob_va = model.predict_proba(X_va)[:, 1]  
    pred_va = (prob_va >= 0.5).astype(int)
```

```

auc = roc_auc_score(y_va, prob_v)
acc = accuracy_score(y_va, pred_va)
f1 = f1_score(y_va, pred_va)
ll = log_loss(y_va, prob_v)

cv_results.append({
    "Fold": fold,
    "AUC": auc,
    "Accuracy": acc,
    "F1": f1,
    "LogLoss": ll
})

print(f"==== Fold {fold} =====")
print(f"AUC: {auc:.6f}")
print(f"Accuracy: {acc:.6f}")
print(f"F1: {f1:.6f}")
print(f"LogLoss: {ll:.6f}")

cv_results_df = pd.DataFrame(cv_results)

print("==== 5-Fold CV Results =====")
print(cv_results_df)

print("==== 5-Fold CV Mean =====")
print(cv_results_df[["AUC", "Accuracy", "F1", "LogLoss"]].mean())

print("==== 5-Fold CV SD =====")
print(cv_results_df[["AUC", "Accuracy", "F1", "LogLoss"]].std())

# =====
# 9. 전체 Train 데이터로 최종 XGBoost 학습
# =====

final_model = make_xgb_model(random_state=42)
final_model.fit(X_train, y_train)

# =====
# 10. 전체 데이터 스코어링
# =====

X_all, _ = preprocess_data(df, train_columns=train_cols)

all_prob_1 = final_model.predict_proba(X_all)[:, 1]
all_prob_0 = 1.0 - all_prob_1

```

```
all_pred = (all_prob_1 >= 0.5).astype(int)
```

```
# =====  
# 11. SAS Viya 출력 데이터 생성  
# =====
```

```
dm_scoreddf = df.copy()
```

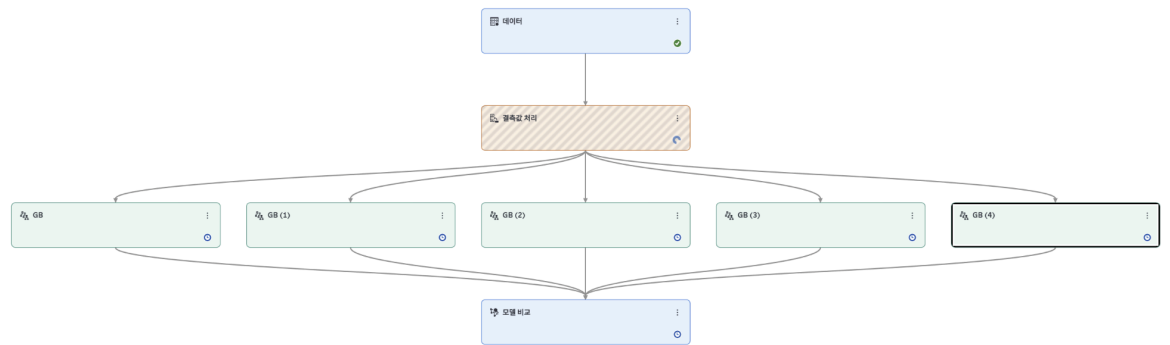
```
dm_scoreddf["P_target_churn1"] = all_prob_1  
dm_scoreddf["P_target_churn0"] = all_prob_0  
dm_scoreddf["I_target_churn"] = all_pred.astype(str)
```

```
print("==== dm_scoreddf head ====")  
print(dm_scoreddf[["P_target_churn0", "P_target_churn1", "I_target_churn"]].head())
```

```
print("==== XGBoost 5-Fold CV + Final Scoring Completed ====")
```

데마전처리-변수선택 선정

1.결측값처리



챔피언	이름	알고리즘...	ROC 아...	정확도	평균제곱...	누적항...	누적반응...
★	GB	그래디언트 부스팅	0.7977	0.9129	0.0712	3.9067	39.0669
	GB (1)	그래디언트 부스팅	0.7971	0.9131	0.0712	3.8786	38.7858
	GB (2)	그래디언트 부스팅	0.7958	0.9134	0.0712	3.8561	38.5610
	GB (3)	그래디언트 부스팅	0.7977	0.9132	0.0710	3.8449	38.4486
	GB (4)	그래디언트 부스팅	0.7963	0.9130	0.0713	3.7943	37.9427

ROC-AUC 값

0.7977

0.7971

0.7958

0.7977

0.7963

평균 (Mean)

0.79692

표준편차 (SD)

0.00085

약 0.0009

2.군집화

모델 비교

챔피언	이름	알고리즘...	ROC 아...	정확도	평균제곱...	누적항...	누적반응...	절사값	데이터 ...
★	GB	그래디언트 부스팅	0.7981	0.9126	0.0715	3.8392	38.3924	0.5000	VALIDATE
	GB (3)	그래디언트 부스팅	0.7962	0.9127	0.0713	3.9741	39.7414	0.5000	VALIDATE
	GB (1)	그래디언트 부스팅	0.7950	0.9128	0.0714	3.9067	39.0669	0.5000	VALIDATE
	GB (4)	그래디언트 부스팅	0.7964	0.9126	0.0719	3.8111	38.1113	0.5000	VALIDATE
	GB (2)	그래디언트 부스팅	0.7978	0.9126	0.0712	3.8561	38.5610	0.5000	VALIDATE

ROC-AUC 값

0.7981

0.7962

0.7950

0.7964

0.7978

평균 (Mean)

0.7967

표준편차 (SD)

0.0013

약 0.0012 ~ 0.0013

3.변수관리

모델 비교

챔피언	이름	알고리즘...	ROC 아...	정확도	평균제곱...	누적항...	누적반응...	절사값	데이터 ...
★	GB	그래디언트 부스팅	0.7977	0.9129	0.0712	3.9067	39.0669	0.5000	VALIDAT
	GB (1)	그래디언트 부스팅	0.7971	0.9131	0.0712	3.8786	38.7858	0.5000	VALIDAT
	GB (2)	그래디언트 부스팅	0.7958	0.9134	0.0712	3.8561	38.5610	0.5000	VALIDAT
	GB (4)	그래디언트 부스팅	0.7963	0.9130	0.0713	3.7943	37.9427	0.5000	VALIDAT
	GB (3)	그래디언트 부스팅	0.7977	0.9132	0.0710	3.8449	38.4486	0.5000	VALIDAT

ROC-AUC 값

0.7977

0.7971

0.7958

0.7963
0.7977

평균 (Mean)
0.79692

표준편차 (SD)
0.00085
약 0.0009

4. 대체

- .

챔피언	이름	알고리즘...	ROC 아...	정확도	평균제곱...	누적항...	누적반응...	절사값	데이
★	GB (3)	그래디언트 부스팅	0.7972	0.9133	0.0712	3.8730	38.7296	0.5000	VAL
	GB	그래디언트 부스팅	0.7960	0.9124	0.0712	3.8898	38.8983	0.5000	VAL
	GB (2)	그래디언트 부스팅	0.7925	0.9128	0.0714	3.8786	38.7858	0.5000	VAL
	GB (4)	그래디언트 부스팅	0.7955	0.9129	0.0713	3.8280	38.2799	0.5000	VAL
	GB (1)	그래디언트 부스팅	0.7950	0.9127	0.0715	3.8730	38.7296	0.5000	VAL

ROC-AUC 값
0.7972
0.7960
0.7925
0.7955
0.7950

평균 (Mean)
0.79524

표준편차 (SD)
0.0018
약 0.0020

5. 변수선택

챔피언	이름	알고리즘...	ROC 아...	정확도	평균제곱...	누적항...	누적반응...
★	GB (3)	그래디언트 부스팅	0.8013	0.9131	0.0708	3.9882	39.8820
	GB (1)	그래디언트 부스팅	0.8007	0.9131	0.0709	3.8617	38.6172
	GB (2)	그래디언트 부스팅	0.8001	0.9134	0.0708	3.8898	38.8983
	GB (4)	그래디언트 부스팅	0.8001	0.9131	0.0709	3.9348	39.3479
	GB	그래디언트 부스팅	0.8010	0.9128	0.0708	3.9067	39.0669

ROC-AUC 값

0.8013

0.8007

0.8001

0.8001

0.8010

평균 (Mean)

0.80064

표준편차 (SD)

0.00053

약 0.0005

6.변환

모델 비교

챔피언	이름	알고리즘...	ROC 아...	정확도	평균제곱...	누적향...	누적반응...	절사값	데이터 ...	중
★	GB	그래디언트 부스팅	0.7977	0.9129	0.0712	3.9067	39.0669	0.5000	VALIDATE	
	GB (4)	그래디언트 부스팅	0.7963	0.9130	0.0713	3.7943	37.9427	0.5000	VALIDATE	
	GB (3)	그래디언트 부스팅	0.7977	0.9132	0.0710	3.8449	38.4486	0.5000	VALIDATE	
	GB (2)	그래디언트 부스팅	0.7958	0.9134	0.0712	3.8561	38.5610	0.5000	VALIDATE	
	GB (1)	그래디언트 부스팅	0.7971	0.9131	0.0712	3.8786	38.7858	0.5000	VALIDATE	

ROC-AUC 값

0.7977

0.7963

0.7977

0.7958

0.7971

평균 (Mean)

0.79692

표준편차 (SD)

0.00085

약 0.0009

변수선택-변수선택 선정 유지

1.Variable Selection 2



ROC-AUC 값

0.7989

0.7984

0.7983

0.7973

0.7986

평균 (Mean)

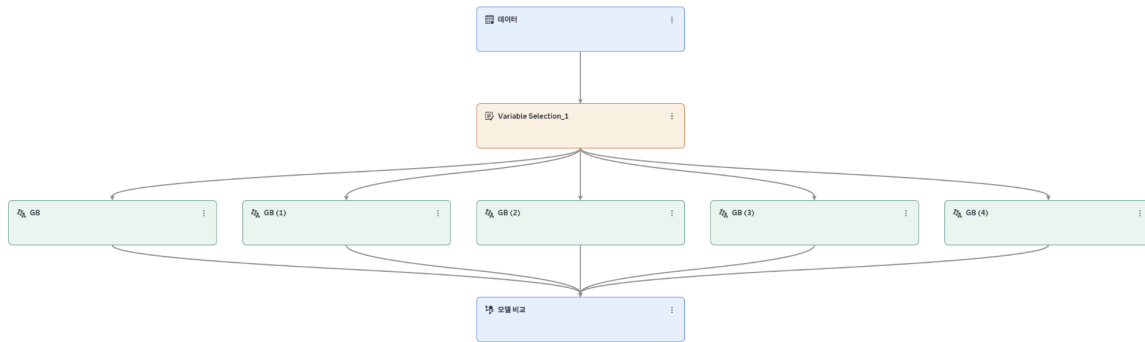
0.7983

표준편차 (SD)

0.0006

약 0.0006 ~ 0.0007

2.Variable Selection_1



챔피언	이름	알고리즘...	ROC 아...	정확도	평균제곱...	누적항...	누적반응...	절사값	E
★	GB (2)	그래디언트 부스팅	0.7989	0.9136	0.0708	3.9236	39.2355	0.5000	\
	GB	그래디언트 부스팅	0.7984	0.9131	0.0711	3.8449	38.4486	0.5000	\
	GB (1)	그래디언트 부스팅	0.7983	0.9128	0.0712	3.9123	39.1231	0.5000	\
	GB (4)	그래디언트 부스팅	0.7973	0.9134	0.0711	3.8336	38.3361	0.5000	\
	GB (3)	그래디언트 부스팅	0.7986	0.9129	0.0709	3.9077	39.0768	0.5000	\

ROC-AUC 값

0.7989

0.7984

0.7983

0.7973

0.7986

평균 (Mean)

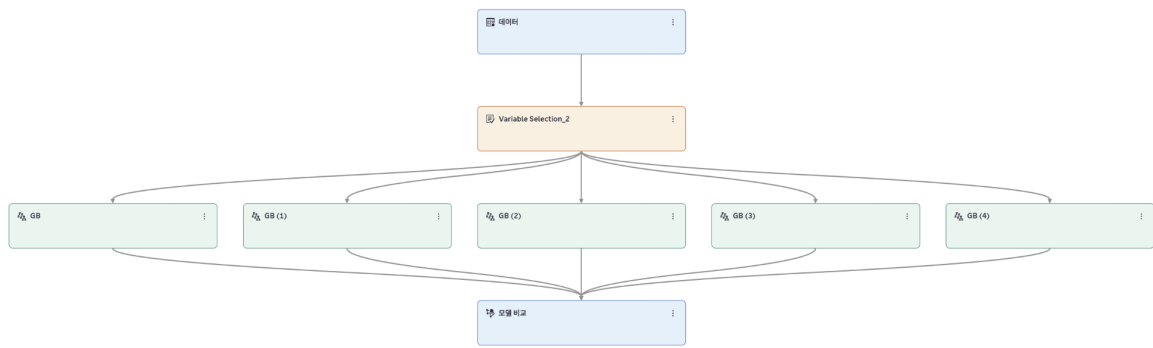
0.79830

표준편차 (SD)

0.00061

약 0.0006

3.Variable Selection_2



챔피언	이름	알고리즘...	ROC 아...	정확도	평균제곱...	누적합...	누적반응...	절사값	데이터 ...	
★	GB (4)	그래디언트 부스팅	0.7729	0.9126	0.0720	3.8456	38.4559	0.5000	VALIDATE	
	GB	그래디언트 부스팅	0.7674	0.9118	0.0721	3.8280	38.2799	0.5000	VALIDATE	
	GB (1)	그래디언트 부스팅	0.7720	0.9123	0.0721	3.8392	38.3924	0.5000	VALIDATE	
	GB (3)	그래디언트 부스팅	0.7694	0.9124	0.0720	3.7437	37.4368	0.5000	VALIDATE	
	GB (2)	그래디언트 부스팅	0.7721	0.9127	0.0721	3.8940	38.9404	0.5000	VALIDATE	

ROC-AUC 값

0.7729

0.7674

0.7720

0.7694

0.7721

평균 (Mean)

0.77076

표준편차 (SD)

0.00243

약 0.0024

양상블

FOREST

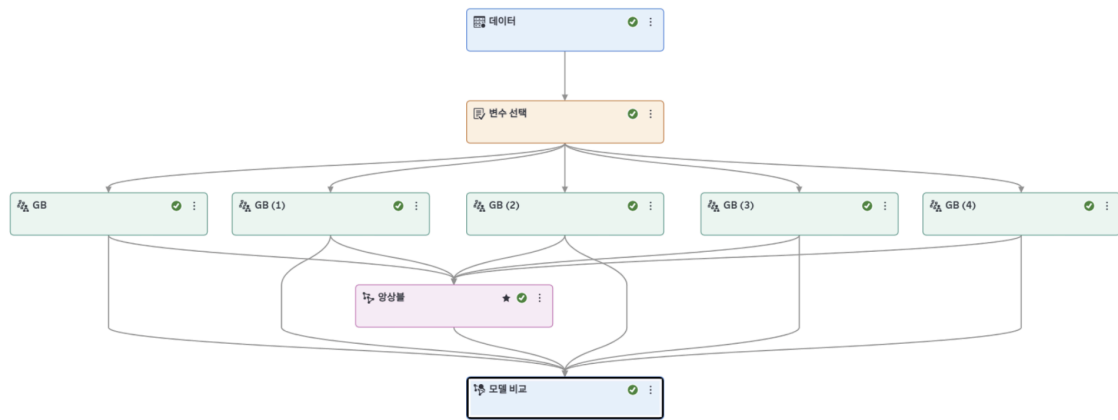
하향평준화

LOGI

신경망

하향평준화

GB



Model Studio - 모델 생성

모델 비교

챔피언	이름	알고리...	ROC 아...	정확도	평균제...	누적향...
★	양상블	양상블	0.8013	0.9131	0.0708	3.9179
	GB (1)	그래디언트 부스팅	0.8007	0.9131	0.0709	3.8617
	GB (2)	그래디언트 부스팅	0.8001	0.9134	0.0708	3.8898
	GB (4)	그래디언트 부스팅	0.8001	0.9131	0.0709	3.9348
	GB (3)	그래디언트 부스팅	0.8013	0.9131	0.0708	3.9882
	GB	그래디언트 부스팅	0.8010	0.9128	0.0708	3.9067

변수선택-GB에 추가

피쳐머신

모델 비교

챔피언	이름	알고리...	ROC 아...	정확도	평균제...	
★	GB (3)	그래디언트 부스팅	0.7959	0.9125	0.0713	
	GB (4)	그래디언트 부스팅	0.7943	0.9130	0.0717	
	GB (2)	그래디언트 부스팅	0.7950	0.9127	0.0714	
	GB	그래디언트 부스팅	0.7939	0.9127	0.0716	
	GB (1)	그래디언트	0.7932	0.9134	0.0712	

하향

대체

하향

변환



LTV 모델

LTV 모델

1. 이탈(Churn) 예측 모델 개발 진행사항

1.1. 데이터 전처리 및 탐색적 데이터 분석 (EDA)

- **데이터 정제:** 데이터 타입 변환 완료.

- **target_ltv** : Target/Interval
- **target_churn** : Rejected (데이터 누수 방지 - 다른 정답 변수 보고 치팅)
- **customer_id** : ID/Nominal
- **net_balance** : input/interval
- **active_months_6m** : input/ interval
- **category_nunique_6m** : input/ interval
- **fin_overdue_days** : input/ interval
- **num_active_cards** : input/ interval
- **region_code** : input/nominal
- **txn_cnt_1m** : input/ interval
- **zero_month_count** : input/interval
- **category_nunique_6m** : input/interval
-

변수명	의미	역할	레벨
active_months_6m	최근 6개월 활동 월 수	Input	Interval
age	연령	Input	Interval
amt_month_slope	월별 거래금액 변화 추세	Input	Interval
card_cash_service_amt_0	카드 현금서비스 금액 처리 변수	Input	Interval
card_cash_service_amt_log	카드 현금서비스 금액 로그 변환	Input	Interval
card_loan_amt_log	카드론 금액 로그 변환	Input	Interval

변수명	의미	역할	레벨
category_nunique_6m	최근 6개월 사용 카테고리 종류 수	Input	Interval
credit_score	신용점수	Input	Interval
customer_id	고객 식별자	ID	Nominal
days_since_last_txn	마지막 거래 이후 경과일	Input	Interval
fin_asset_trend_score	금융자산 추세 점수	Input	Interval
fin_overdue_days	금융 연체일 수	Input	Interval
gender	성별	Input	Binary
income_group	소득 구간	Input	Nominal
installment_ratio_6m	최근 6개월 할부 거래 비율	Input	Interval
is_married	결혼 여부	Input	Nominal
loan_to_deposit_cut	예금 대비 대출 비율 처리 변수	Input	Interval
net_balance	순잔고	Input	Interval
num_active_cards	활성 카드 수	Input	Interval
online_ratio_6m	최근 6개월 온라인 거래 비율	Input	Interval
prefer_category	선호 소비 카테고리	Input	Nominal
recent_amt_share_1m	최근 1개월 거래금액 비중	Input	Interval
recent_amt_share_3m	최근 3개월 거래금액 비중	Input	Interval
region_code	거주/지역 코드	Input	Nominal

변수명	의미	역할	레벨
target_churn	고객 이탈 여부	Rejected	Binary
target_ltv	고객 생애가치 원 변수	Target	Interval
target_ltv_log	고객 생애가치 로그 변환 변수	Rejected	Interval
tenure_days	고객 유지 기간	Input	Interval
total_deposit_balance_log	총 예금잔액 로그 변환	Input	Interval
total_loan_balance_log	총 대출잔액 로그 변환	Input	Interval
txn_amt_mean_6m	최근 6개월 평균 거래금액	Input	Interval
txn_amt_std_6m	최근 6개월 거래금액 표준편차	Input	Interval
txn_amt_sum_1m	최근 1개월 총 거래금액	Input	Interval
txn_amt_sum_3m	최근 3개월 총 거래금액	Input	Interval
txn_amt_sum_6m	최근 6개월 총 거래금액	Input	Interval
txn_cnt_1m	최근 1개월 거래건수	Input	Interval
txn_cnt_3m	최근 3개월 거래건수	Input	Interval
txn_cnt_6m	최근 6개월 거래건수	Input	Interval

변수 이름	↑	레이블	유형	역할	변형 평가	레벨
active_months_6m	☐		숫자	Input		Interval
age	☐		숫자	Input		Interval
amt_month_slope	☐		숫자	Input		Interval
card_cash_service_amt	☐		숫자	Input		Interval
card_loan_amt	☐		숫자	Input		Interval
category_nunique_6m	☐		숫자	Input		Interval
credit_score	☐		숫자	Input		Interval
customer_id	☐		문자	ID		Nominal
days_since_last_txn	☐		숫자	Input		Interval
fin_asset_trend_score	☐		숫자	Input		Interval
fin_overdue_days	☐		숫자	Input		Interval
gender	☐		문자	Input		Binary
income_group	☐		문자	Input		Nominal
installment_ratio_6m	☐		숫자	Input		Interval
is_married	☐		숫자	Input		Binary
loan_to_deposit	☐		숫자	Input		Interval
net_balance	☐		숫자	Input		Interval
num_active_cards	☐		숫자	Input		Interval
online_ratio_6m	☐		숫자	Input		Interval
overdue_flag	☐		숫자	Input		Binary
prefer_category	☐		문자	Input		Nominal
recent_amt_share_1m	☐		숫자	Input		Interval
recent_amt_share_3m	☐		숫자	Input		Interval
region_code	☐		문자	Input		Nominal
target_churn	☐		숫자	Rejected		Binary
target_ltv	☐		숫자	Target		Interval
tenure_days	☐		숫자	Input		Interval
total_deposit_balance	☐		숫자	Input		Interval
total_loan_balance	☐		숫자	Input		Interval
txn_amt_mean_6m	☐		숫자	Input		Interval
txn_amt_std_6m	☐		숫자	Input		Interval
txn_amt_sum_1m	☐		숫자	Input		Interval
txn_amt_sum_3m	☐		숫자	Input		Interval
txn_amt_sum_6m	☐		숫자	Input		Interval
txn_cnt_1m	☐		숫자	Input		Interval
txn_cnt_3m	☐		숫자	Input		Interval
txn_cnt_6m	☐		숫자	Input		Interval
zero_month_count	☐		숫자	Input		Interval

1.2 프로젝트 설정

- 데이터 분할 - 70 : 30
- 규칙 - Interval 모델 선택 통계량: 제공근 평균제공오차

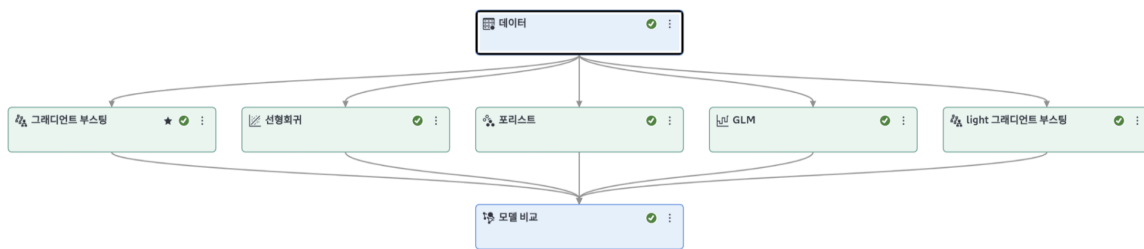
+

light GBM

1. 비선형을 잘 잡아
2. 변수들 사이 상호작용을 잘 잡아

2. 모델 탐색

2.1 알고리즘 선정



모형 비교

챔피언	이름	알고리즘 이름	제공근평균제곱오차
★	그래디언트 부스팅	그래디언트 부스팅	1,342,648.1871
	선형회귀	선형회귀	1,348,888.7130
	포리스트	포리스트	1,348,625.7660
	GLM	GLM	1,357,000.6049
	light 그래디언트 부스팅	LightGBM	1,344,167.6032

결론. 그래디언트 부스팅 1,342,648.1871 로 가자.

2.2 하이퍼파라미터 튜닝

실험 A

그래디언트 부스팅 튜닝

- learning rate 조금 낮추기
- 트리 수 늘리기
- 깊이 약간 줄이기 또는 leaf 관련 규제 강화

실험 B

LightGBM 튜닝

- leaf 수 줄이기
- min data in leaf 키우기
- learning rate 낮추기
- 과적합 줄이는 방향으로

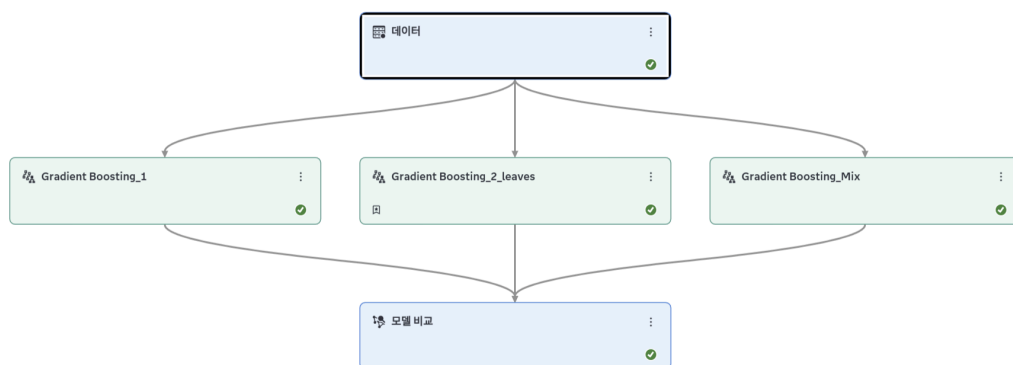
실험 C

타겟 변환 버전

- $\log(LTV + 1)$ 또는 로그변환된 타겟으로 별도 모델
- 예측 후 다시 역변환
- LTV가 오른쪽 꼬리가 길면 이게 꽤 잘 먹힐 수 있어

결론. 데이터 마이닝 사전 처리로 가자.

2.3 GB 디폴트 모델 비교

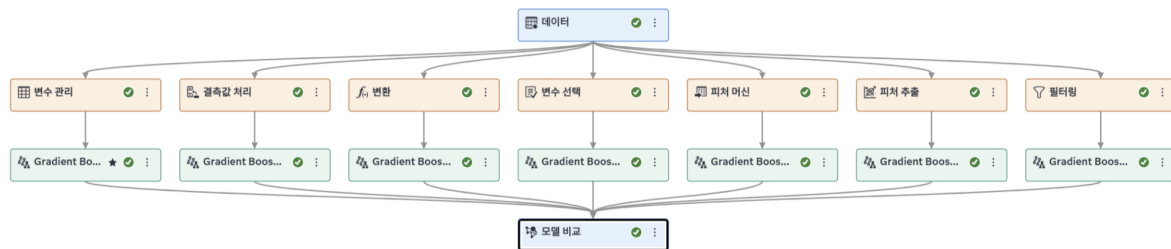


모델 비교

챔피언	이름	알고리즘 이름	제공근평균제공오차	평균제공오차	데이터 역할
★	Gradient Boosting_2_leaves	그래디언트 부스팅	1,342,107.9654	1,801,253,790,650	VALIDATE
	Gradient Boosting_1	그래디언트 부스팅	1,350,421.9881	1,823,639,545,980	VALIDATE
	Gradient Boosting_Mix	그래디언트 부스팅	1,344,857.4272	1,808,641,499,450	VALIDATE

결론. Gradient Boosting_2_leaves 1,342,107.9654 로 가자.

2.4 데이터 마이닝 사전처리



챔피언	이름	알고리즘 이름	제공근평균제곱오차	데이터 역할
★	Gradient Boosting_2_leaves	그래디언트 부스팅	1,342,107.9654	VALIDATE
	Gradient Boosting_2_leaves (1)	그래디언트 부스팅	1,342,107.9654	VALIDATE
	Gradient Boosting_2_leaves (2)	그래디언트 부스팅	1,342,107.9654	VALIDATE
	Gradient Boosting_2_leaves (3)	그래디언트 부스팅	1,347,893.1103	VALIDATE
	Gradient	그래디언트 부스팅	1,344,291.4965	VALIDATE

결론. 변수관리 + Gradient Boosting_2_leaves 1,342,107.9654 .

- 현재 해야할 단계(데이터 전처리 - 빈)
 1. 지금 churn의 GB_Mix와 LTV의 GB_2_leaves는 일단 임시 best
 2. 정제된 변수로 churn: GB_Mix, RF, Logistic 와 LTV: GB_2_leaves, RF 등 다시 순위 매기기 [가능하면 XGB와 Cat Boost도 같이 돌려보기]
 3. LTV는 $\log_{1p}(\text{target_ltv})$ 버전도 반드시 비교
- 현재 해야할 단계 (SAS 머신러닝 - 헤리)
 - A. split (split seed or random_state)
 - B. n-fold

둘 중 하나 사용해서 모델 성능 비교 다시 해보기. 최적의 모델은 여러 번 돌려가며 찾아야 한다.

D. 튜닝이 무거우면 “축소 튜닝”만 하세요

전면 grid search 말고,
정말 핵심 파라미터만 적게 보시면 됩니다.

예를 들어 GB에서는 보통:

- tree depth
- learning rate
- number of trees / iterations
- minimum leaf size 정도

만 좁은 범위로 보세요.

즉:

- depth 3, 5, 7
- learning rate 0.03, 0.05, 0.1
- trees 100, 200, 300

이런 식의 작은 탐색이면 훨씬 낫습니다.

지금은 “정교한 최적화”보다
거칠게라도 한 번 튜닝 효과를 확인하는 것이 더 중요합니다.

-

```

import pandas as pd
import numpy as np
from xgboost import XGBRegressor
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

# 1. 데이터 로드
df = dm_inputdf.copy()

# 2. 전처리 함수: 원본 데이터 최적화
def preprocess_direct(input_df, train_columns=None):
    # 제외 컬럼 (Leakage 방지)
    drop_list = ["target_ltv", "target_ltv_log", "target_churn", "customer_id",
                 "_dmIndex_", "_PartInd_"]
    X = input_df.drop(columns=[c for c in drop_list if c in input_df.columns],
                       errors="ignore")

    # [정제] 입력 수치형 변수 95% Capping (폭주 방지 핵심)
    num_cols = X.select_dtypes(include=[np.number]).columns
    for col in num_cols:
        upper_limit = X[col].quantile(0.95)
        X[col] = X[col].clip(upper=upper_limit)

    # [정제] 범주형 변수 처리
    cat_cols = ["gender", "is_married", "income_group", "prefer_category",
                 "region_code", "overdue_flag"]
    X_processed = pd.get_dummies(X, columns=[c for c in cat_cols if c in X.columns],
                                  drop_first=True)

    # 컬럼 일치화 (에러 방지)
    if train_columns is not None:
        X_processed = X_processed.reindex(columns=train_columns, fill_value=0)

    return X_processed.fillna(0)

# 3. 데이터 분할 및 타겟 설정
train_df = df[df["_PartInd_"] == 1].copy()
valid_df = df[df["_PartInd_"] == 0].copy()

# 타겟 설정: 원본 ltv를 그대로 사용 (단, 학습 안정성을 위해 95% Capping만 적용)
target_upper_limit = train_df["target_ltv"].quantile(0.95)
y_train = train_df["target_ltv"].clip(upper=target_upper_limit)
y_valid = valid_df["target_ltv"].clip(upper=target_upper_limit)

# 데이터 준비
X_train = preprocess_direct(train_df)

```

```

train_cols = X_train.columns
X_valid = preprocess_direct(valid_df, train_columns=train_cols)

# 4. XGBoost 모델 학습 (원본 금액 학습용 세팅)
model = XGBRegressor(
    n_estimators=800,
    learning_rate=0.03,
    max_depth=5,
    reg_alpha=1,          # L1 규제 추가 (불필요한 변수 영향 감소)
    reg_lambda=10,        # L2 규제 강화
    objective="reg:squarederror",
    random_state=42
)
model.fit(X_train, y_train)

# 5. 예측
y_pred = model.predict(X_valid)

# 6. 최종 성능 리포트
rmse = np.sqrt(mean_squared_error(y_valid, y_pred))
mae = mean_absolute_error(y_valid, y_pred)
r2 = r2_score(y_valid, y_pred)

print("\n" + "="*50)
print("    [LTV Direct Model] 원본 금액 모델 성능 리포트")
print("="*50)
print(f"▶ RMSE (오차 금액): {rmse:,.2f}")
print(f"▶ MAE (평균 오차): {mae:,.2f}")
print(f"▶ R2 Score (설명력): {r2:,.4f}")
print("-" * 50)
print(f"※ 상한선 기준: {target_upper_limit:,.0f}원 (95% Capping)")
print("="*50)

# 7. SAS 결과 저장
X_all = preprocess_direct(df, train_columns=train_cols)
dm_scoreddf = df.copy()
dm_scoreddf["P_target_ltv"] = model.predict(X_all)

```

● 결과값

```

=====
[LTV Direct Model] 원본 금액 모델 성능 리포트
=====
▶ RMSE (오차 금액): 1,113,987.69

```

▶ MAE (평균 오차) : 898,068.82

▶ R2 Score (설명력) : 0.0112

※ 상한선 기준: 3,931,566원 (95% Capping)

=====

파이썬 first LTV

1. 데이터 타입

- **target_ltv** : Target/Interval
- **target_churn** : Rejected (데이터 누수 방지 - 다른 정답 변수 보고 치팅)
- **customer_id** : ID/Nominal
- **net_balance** : input/interval
- **active_months_6m** : input/ interval
- **category_nunique_6m** : input/ interval
- **fin_overdue_days** : input/ interval
- **num_active_cards** : input/ interval
- **region_code** : input/nominal
- **txn_cnt_1m** : input/ interval
- **zero_month_count** : input/interaval
- **category_nunique_6m** : input/interval
- **card_cash_service_amt_0** : input/interval

문제. sas viya에서 ltv log reject하면 다른 파이프라인 오픈소스코드에 ltv log볼러올 수 있는지.

GB-캐핑o

결과

```
[Fold 1] RMSE: 1,373,178.57 / MAE: 996,859.73 / R2: 0.0040
[Fold 2] RMSE: 1,347,779.14 / MAE: 982,847.02 / R2: 0.0070
[Fold 3] RMSE: 1,339,317.57 / MAE: 984,562.19 / R2: 0.0003
[Fold 4] RMSE: 1,365,519.09 / MAE: 986,814.70 / R2: 0.0044
[Fold 5] RMSE: 1,362,475.36 / MAE: 993,660.50 / R2: 0.0010
```

```
=====
[5-Fold CV 결과] LTV Model - GB + X Capping Only
=====
```

RMSE 평균: 1,357,653.95

RMSE 표준편차: 12,327.65

MAE 평균: 988,948.83

R2 평균: 0.0034

```
=====
      P_target_ltv
0  1.122130e+06
1  1.182736e+06
2  1.259229e+06
3  1.118913e+06
4  8.964411e+05
```

코드

GB-캐핑 x

결과

```
[Fold 1] RMSE: 1,376,091.98 / MAE: 997,779.23 / R2: -0.0002
[Fold 2] RMSE: 1,352,634.20 / MAE: 985,899.93 / R2: -0.0002
[Fold 3] RMSE: 1,342,005.04 / MAE: 986,271.51 / R2: -0.0037
[Fold 4] RMSE: 1,364,577.95 / MAE: 985,679.88 / R2: 0.0058
[Fold 5] RMSE: 1,364,208.86 / MAE: 992,968.96 / R2: -0.0015
```

```
=====
[5-Fold CV 결과] LTV Model - GB No Capping
=====
```

```
RMSE 평균: 1,359,903.60
```

```
RMSE 표준편차: 11,624.55
```

```
MAE 평균: 989,719.90
```

```
R2 평균: 0.0000
```

```
=====
P_target_ltv
0  1.274485e+06
1  1.128235e+06
2  1.225300e+06
3  1.160447e+06
4  1.007339e+06
```

코드



XGB-캐핑o

#결과

```
[Fold 1] RMSE: 1,372,611.12 / MAE: 995,385.34 / R2: 0.0048
[Fold 2] RMSE: 1,349,040.71 / MAE: 982,609.46 / R2: 0.0051
[Fold 3] RMSE: 1,336,856.65 / MAE: 981,629.51 / R2: 0.0040
[Fold 4] RMSE: 1,364,592.50 / MAE: 986,111.83 / R2: 0.0058
[Fold 5] RMSE: 1,358,921.41 / MAE: 991,104.32 / R2: 0.0062
```

```
=====
[5-Fold CV 결과] XGBoost LTV - X Capping Only
=====
RMSE 평균: 1,356,404.48
RMSE 표준편차: 12,428.81
MAE 평균: 987,368.09
R2 평균: 0.0052
=====
```

#코드

```
import pandas as pd
import numpy as np
```

```
from xgboost import XGBRegressor
from sklearn.model_selection import KFold
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

1. 데이터 로드

```
df = dm_inputdf.copy()
```

2. 전처리 함수: X 변수만 capping

```
def preprocess_x_capping(input_df, train_columns=None, cap_limits=None):
```

```
    drop_list = [
        "target_ltv", "target_ltv_log", "target_churn",
        "customer_id", "_dmIndex_", "_PartInd_"
    ]
```

```
    X = input_df.drop(
        columns=[c for c in drop_list if c in input_df.columns],
        errors="ignore"
    ).copy()
```

수치형 입력변수만 95% capping

```
num_cols = X.select_dtypes(include=[np.number]).columns
```

```
if cap_limits is None:
```

```
    cap_limits = {}
```

```
    for col in num_cols:
```

```

        cap_limits[col] = X[col].quantile(0.95)

for col, upper_limit in cap_limits.items():
    if col in X.columns:
        X[col] = X[col].clip(upper=upper_limit)

# 범주형 변수 더미 처리
cat_cols = ["gender", "income_group", "prefer_category", "region_code"]

X_processed = pd.get_dummies(
    X,
    columns=[c for c in cat_cols if c in X.columns],
    drop_first=True
)

# train/valid 컬럼 일치
if train_columns is not None:
    X_processed = X_processed.reindex(columns=train_columns, fill_value=0)

return X_processed.fillna(0), cap_limits

```

```

# 3. CV 대상 데이터
data = df[df["target_ltv"].notna()].copy()

```

```

# 4. 5-Fold CV 설정
kf = KFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

```

```

rmse_list = []
mae_list = []
r2_list = []

```

```

# 5. 5-Fold CV 실행
for fold, (train_idx, valid_idx) in enumerate(kf.split(data), start=1):
    train_df = data.iloc[train_idx].copy()
    valid_df = data.iloc[valid_idx].copy()

    X_train, cap_limits = preprocess_x_capping(train_df)
    train_cols = X_train.columns

    X_valid, _ = preprocess_x_capping(
        valid_df,

```

```

        train_columns=train_cols,
        cap_limits=cap_limits
    )

    y_train = train_df["target_ltv"]
    y_valid = valid_df["target_ltv"]

    model = XGBRegressor(
        n_estimators=800,
        learning_rate=0.03,
        max_depth=5,
        reg_alpha=1,
        reg_lambda=10,
        objective="reg:squarederror",
        random_state=42,
        n_jobs=1
    )

    model.fit(X_train, y_train)

    y_pred = model.predict(X_valid)
    y_pred = np.maximum(y_pred, 0)

    rmse = np.sqrt(mean_squared_error(y_valid, y_pred))
    mae = mean_absolute_error(y_valid, y_pred)
    r2 = r2_score(y_valid, y_pred)

    rmse_list.append(rmse)
    mae_list.append(mae)
    r2_list.append(r2)

    print(f"[Fold {fold}] RMSE: {rmse:,.2f} / MAE: {mae:,.2f} / R2: {r2:,.4f}")

```

6. CV 결과 요약

```

print("\n" + "="*60)
print("      [5-Fold CV 결과] XGBoost LTV - X Capping Only")
print("="*60)
print(f"RMSE 평균: {np.mean(rmse_list):,.2f}")
print(f"RMSE 표준편차: {np.std(rmse_list):,.2f}")
print(f"MAE 평균: {np.mean(mae_list):,.2f}")
print(f"R2 평균: {np.mean(r2_list):,.4f}")
print("="*60)

```

7. 전체 train 데이터로 최종 모델 재학습

```

X_final, final_cap_limits = preprocess_x_capping(data)
final_cols = X_final.columns

```

```
y_final = data["target_ltv"]
```

```
final_model = XGBRegressor(  
    n_estimators=800,  
    learning_rate=0.03,  
    max_depth=5,  
    reg_alpha=1,  
    reg_lambda=10,  
    objective="reg:squarederror",  
    random_state=42,  
    n_jobs=1  
)
```

```
final_model.fit(X_final, y_final)
```

8. SAS 결과 저장

```
X_score, _ = preprocess_x_capping(  
    df,  
    train_columns=final_cols,  
    cap_limits=final_cap_limits  
)
```

```
dm_scoreddf = df.copy()  
dm_scoreddf["P_target_ltv"] = np.maximum(final_model.predict(X_score), 0)
```

XGB-캐핑 x

#결과

```
[Fold 1] RMSE: 1,374,182.65 / MAE: 1,015,831.79 / R2: -0.0484
[Fold 2] RMSE: 1,364,965.74 / MAE: 1,000,772.07 / R2: -0.0317
[Fold 3] RMSE: 1,455,595.64 / MAE: 1,031,017.05 / R2: -0.0309
[Fold 4] RMSE: 1,357,241.94 / MAE: 980,087.03 / R2: -0.0280
[Fold 5] RMSE: 1,436,043.01 / MAE: 1,031,581.71 / R2: -0.0312
```

```
=====
[5-Fold CV 결과] XGBoost LTV - No Capping
=====
RMSE 평균: 1,397,605.80
RMSE 표준편차: 40,208.21
MAE 평균: 1,011,857.93
R2 평균: -0.0340
=====
```

#코드

```
import pandas as pd
import numpy as np

from xgboost import XGBRegressor
from sklearn.model_selection import KFold
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# 1. 데이터 로드
df = dm_inputdf.copy()

# 2. 전처리 함수: Capping 없음
def preprocess_no_capping(input_df, train_columns=None):
    drop_list = [
        "target_ltv", "target_ltv_log", "target_churn",
        "customer_id", "_dmIndex_", "_PartInd_"
    ]

    X = input_df.drop(
        columns=[c for c in drop_list if c in input_df.columns],
        errors="ignore"
    ).copy()

    # 범주형 변수 더미 처리
    cat_cols = ["gender", "income_group", "prefer_category", "region_code"]

    X_processed = pd.get_dummies(
```

```

X,
columns=[c for c in cat_cols if c in X.columns],
drop_first=True
)

# train/valid 컬럼 일치
if train_columns is not None:
    X_processed = X_processed.reindex(columns=train_columns, fill_value=0)

return X_processed.fillna(0)

```

```

# 3. CV 대상 데이터
data = df[df["target_ltv"].notna()].copy()

```

```

# 4. 5-Fold CV 설정
kf = KFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

```

```

rmse_list = []
mae_list = []
r2_list = []

```

```

# 5. 5-Fold CV 실행
for fold, (train_idx, valid_idx) in enumerate(kf.split(data), start=1):
    train_df = data.iloc[train_idx].copy()
    valid_df = data.iloc[valid_idx].copy()

    X_train = preprocess_no_capping(train_df)
    train_cols = X_train.columns

    X_valid = preprocess_no_capping(
        valid_df,
        train_columns=train_cols
    )

    y_train = train_df["target_ltv"]
    y_valid = valid_df["target_ltv"]

    model = XGBRegressor(
        n_estimators=800,
        learning_rate=0.03,
        max_depth=5,

```

```

        reg_alpha=1,
        reg_lambda=10,
        objective="reg:squarederror",
        random_state=42,
        n_jobs=1
    )

    model.fit(X_train, y_train)

    y_pred = model.predict(X_valid)
    y_pred = np.maximum(y_pred, 0)

    rmse = np.sqrt(mean_squared_error(y_valid, y_pred))
    mae = mean_absolute_error(y_valid, y_pred)
    r2 = r2_score(y_valid, y_pred)

    rmse_list.append(rmse)
    mae_list.append(mae)
    r2_list.append(r2)

    print(f"[Fold {fold}] RMSE: {rmse:,.2f} / MAE: {mae:,.2f} / R2: {r2:,.4f}")

```

6. CV 결과 요약

```

print("\n" + "="*60)
print("    [5-Fold CV 결과] XGBoost LTV - No Capping")
print("="*60)
print(f"RMSE 평균: {np.mean(rmse_list):,.2f}")
print(f"RMSE 표준편차: {np.std(rmse_list):,.2f}")
print(f"MAE 평균: {np.mean(mae_list):,.2f}")
print(f"R2 평균: {np.mean(r2_list):,.4f}")
print("="*60)

```

7. 전체 train 데이터로 최종 모델 재학습

```

X_final = preprocess_no_capping(data)
final_cols = X_final.columns
y_final = data["target_ltv"]

final_model = XGBRegressor(
    n_estimators=800,
    learning_rate=0.03,
    max_depth=5,
    reg_alpha=1,
    reg_lambda=10,
    objective="reg:squarederror",
    random_state=42,
    n_jobs=1
)

```

```
)
```

```
final_model.fit(X_final, y_final)
```

```
# 8. SAS 결과 저장
```

```
X_score = preprocess_no_capping(  
    df,  
    train_columns=final_cols  
)
```

```
dm_scoreddf = df.copy()
```

```
dm_scoreddf["P_target_ltv"] = np.maximum(final_model.predict(X_score), 0)
```

파이썬 first LOG LTV

1. 데이터 타입

- **target_ltv_log** : Target/Interval
- **target_ltv** : rejected
- **target_churn** : Rejected (데이터 누수 방지 - 다른 정답 변수 보고 치팅)
- **customer_id** : ID/Nominal
- **net_balance** : input/interval
- **active_months_6m** : input/ interval
- **category_nunique_6m** : input/ interval
- **fin_overdue_days** : input/ interval
- **num_active_cards** : input/ interval
- **region_code** : input/nominal
- **txn_cnt_1m** : input/ interval
- **zero_month_count** : input/interaval
- **category_nunique_6m** : input/interval
- **card_cash_service_amt_0**: input/interval

LOG GB-캐핑o

```
[Fold 1] RMSE: 1,441,109.36 / MAE: 927,291.49 / R2: -0.2027
[Fold 2] RMSE: 1,486,367.44 / MAE: 931,614.21 / R2: -0.2066
[Fold 3] RMSE: 1,490,571.45 / MAE: 942,677.90 / R2: -0.2285
[Fold 4] RMSE: 1,470,876.11 / MAE: 940,295.17 / R2: -0.2183
[Fold 5] RMSE: 1,563,360.05 / MAE: 970,680.24 / R2: -0.2081
```

```
=====
[5-Fold CV 결과] target_ltv_log → expm1 복원 평가 + X 95% Capping
=====
```

RMSE 평균: 1,490,456.88

RMSE 표준편차: 40,373.74

MAE 평균: 942,511.80

R2 평균: -0.2129

```
=====
```

LOG GB-캐핑x

```
[Fold 1] RMSE: 1,507,901.49 / MAE: 937,299.46 / R2: -0.2010
[Fold 2] RMSE: 1,490,245.03 / MAE: 939,938.93 / R2: -0.2141
[Fold 3] RMSE: 1,471,077.87 / MAE: 925,165.21 / R2: -0.2060
[Fold 4] RMSE: 1,501,003.32 / MAE: 934,069.07 / R2: -0.2029
[Fold 5] RMSE: 1,500,992.41 / MAE: 940,243.69 / R2: -0.2124
```

```
=====
[5-Fold CV 결과] target_ltv_log → expm1 복원 평가
=====
```

```
RMSE 평균: 1,494,244.02
```

```
RMSE 표준편차: 12,887.34
```

```
MAE 평균: 935,343.27
```

```
R2 평균: -0.2073
=====
```

LOG XGB-캐핑o

```
[Fold 1] RMSE: 1,530,546.92 / MAE: 981,701.62 / R2: -0.2357
[Fold 2] RMSE: 1,507,679.85 / MAE: 953,869.98 / R2: -0.2116
[Fold 3] RMSE: 1,510,324.02 / MAE: 957,462.35 / R2: -0.2139
[Fold 4] RMSE: 1,495,185.28 / MAE: 950,061.37 / R2: -0.2070
[Fold 5] RMSE: 1,463,486.62 / MAE: 935,894.31 / R2: -0.2159
```

```
=====
[5-Fold CV 결과] XGB + target_ltv_log → expm1 복원 평가 + X 95% Capping
=====
```

RMSE 평균: 1,501,444.54

RMSE 표준편차: 22,111.78

MAE 평균: 955,797.93

R2 평균: -0.2168

```
=====
```

LOG XGB-캐핑x

[Fold 1] RMSE: 1,504,955.55 / MAE: 938,733.66 / R2: -0.1963
[Fold 2] RMSE: 1,489,357.16 / MAE: 940,845.09 / R2: -0.2126
[Fold 3] RMSE: 1,468,782.84 / MAE: 926,871.26 / R2: -0.2023
[Fold 4] RMSE: 1,500,606.91 / MAE: 935,874.73 / R2: -0.2023
[Fold 5] RMSE: 1,499,575.02 / MAE: 941,447.86 / R2: -0.2101

=====

[5-Fold CV 결과] target_ltv_log → expm1 복원 평가

=====

RMSE 평균: 1,492,655.49

RMSE 표준편차: 12,985.98

MAE 평균: 936,754.52

R2 평균: -0.2047

=====

노드 LTV

2leaves-best

- 1,342,695.4175
- 1,343,192.6750
- 1,343,335.6165
- 1,343,147.2555
- 1,342,753.0729

👉 평균 $\approx$ **1,343,024.81**

_1

모델 비교

챔피언	이름	알고리즘 이름	제공근평균제곱오차	평균제곱오차
★	GB_1 (1)	그래디언트 부스팅	1,342,935.2878	1,803,475,187,240
	GB_1	그래디언트 부스팅	1,343,187.9833	1,804,153,958,470
	GB_1 (2)	그래디언트 부스팅	1,343,325.3480	1,804,522,990,520
	GB_1 (3)	그래디언트 부스팅	1,342,994.5476	1,803,634,354,940
	GB_1 (4)	그래디언트 부스팅	1,343,569.7161	1,805,179,581,930

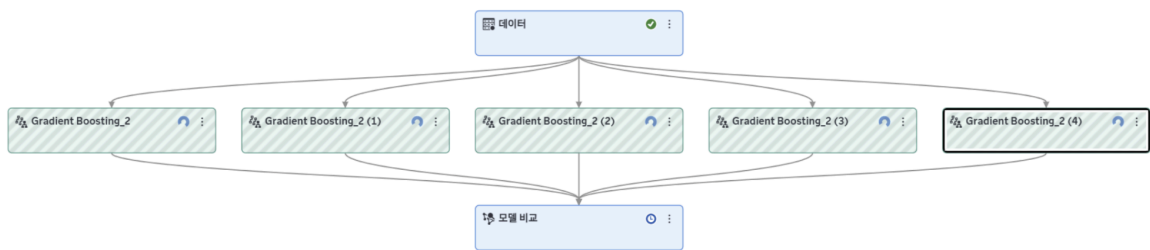
값들 (RMSE):

- 1,342,935.2878
- 1,343,187.9833
- 1,343,325.3480
- 1,342,994.5476
- 1,343,569.7161

👉 평균:

≈ 1,343,202.58

_2



"모델 비교" 결과

노드 평가

모델 비교

챔피언	이름	알고리즘 이름	제공근평균제공오차	평균제공오차	데이터 역할
★	Gradient Boosting_2	그래디언트 부스팅	1,345,326.7360	1,809,904,026,510	VALIDATE
	Gradient Boosting_2 (1)	그래디언트 부스팅	1,346,169.1071	1,812,171,264,800	VALIDATE
	Gradient Boosting_2 (2)	그래디언트 부스팅	1,346,680.9368	1,813,549,545,410	VALIDATE
	Gradient Boosting_2 (3)	그래디언트 부스팅	1,345,349.3900	1,809,964,981,220	VALIDATE
	Gradient Boosting_2 (4)	그래디언트 부스팅	1,345,843.4439	1,811,294,575,580	VALIDATE

폐기

MIX



모델 비교					
챔피언	이름	알고리즘 이름	제공된 평균 제곱 오차	평균 제곱 오차	데이터
★	Gradient Boosting_Mix (2)	그래디언트 부스팅	1,343,608.7058	1,805,284,354,390	VALID
	Gradient Boosting_Mix	그래디언트 부스팅	1,343,663.8411	1,805,432,517,790	VALID
	Gradient Boosting_Mix (1)	그래디언트 부스팅	1,344,148.4303	1,806,735,002,710	VALID
	Gradient Boosting_Mix (3)	그래디언트 부스팅	1,343,771.4162	1,805,721,619,000	VALID
	Gradient Boosting_Mix (4)	그래디언트 부스팅	1,343,617.6192	1,805,308,306,610	VALID

값들:

- 1,343,608.7058
- 1,343,663.8411
- 1,344,148.4303
- 1,343,771.4162
- 1,343,617.6192

👉 평균:

≈ 1,343,762.00

GB.

모델 비교 결과

노드 평가

모델 비교

챔피언	이름	알고리즘 이름	제공근평균제공오차	평균제공오차	데이터 역할	
★	GB (1)	그래디언트 부스팅	1,342,935.2878	1,803,475,187,240	VALIDATE	
	GB	그래디언트 부스팅	1,343,187.9833	1,804,153,958,470	VALIDATE	
	GB (2)	그래디언트 부스팅	1,343,325.3480	1,804,522,990,520	VALIDATE	
	GB (3)	그래디언트 부스팅	1,342,994.5476	1,803,634,354,940	VALIDATE	
	GB (4)	그래디언트 부스팅	1,343,569.7161	1,805,179,581,930	VALIDATE	

값들:

- 1,342,935.2878
- 1,343,187.9833
- 1,343,325.3480
- 1,342,994.5476
- 1,343,569.7161

👉 평균:

≈ 1,343,202.58

GB_1

모델 비교

챔피언	이름	알고리즘 이름	제공된평균제곱오차	평균제곱오차
★	GB_1 (1)	그래디언트 부스팅	1,342,935.2878	1,803,475,187,240
	GB_1	그래디언트 부스팅	1,343,187.9833	1,804,153,958,470
	GB_1 (2)	그래디언트 부스팅	1,343,325.3480	1,804,522,990,520
	GB_1 (3)	그래디언트 부스팅	1,342,994.5476	1,803,634,354,940
	GB_1 (4)	그래디언트 부스팅	1,343,569.7161	1,805,179,581,930

1,342,935.2878

- 1,343,187.9833
- 1,343,325.3480
- 1,342,994.5476
- 1,343,569.7161

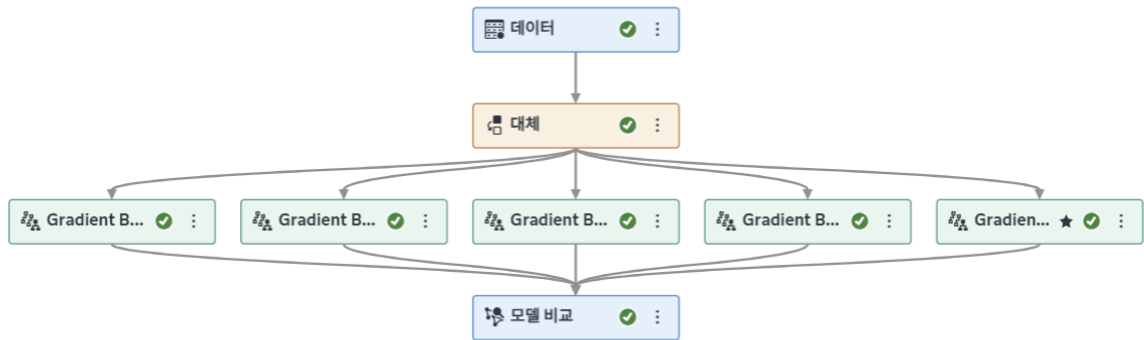
👉 평균:

≈ 1,343,202.58

노드 LTV 구체화

L

대체,



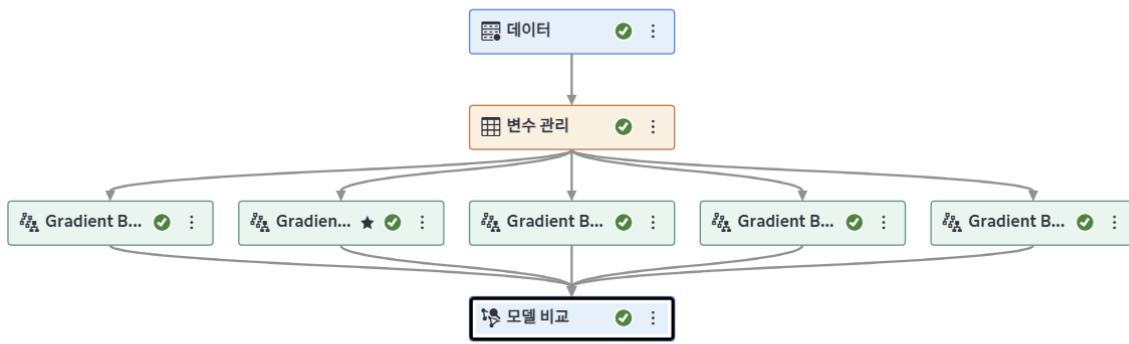
표본 기표

챔피언	이름	알고리즘 이름	제공된평균제공오차	평균?
★	Gradient Boosting_2_leaves (4)	그래디언트 부스팅	1,342,852.3903	1,803,2
	Gradient Boosting_2_leaves (3)	그래디언트 부스팅	1,342,957.4827	1,803,5
	Gradient Boosting_2_leaves (2)	그래디언트 부스팅	1,343,657.2478	1,805,4
	Gradient Boosting_2_leaves	그래디언트 부스팅	1,344,053.4040	1,806,4
	Gradient Boosting_2_leaves (1)	그래디언트 부스팅	1,343,177.4857	1,804,1

평균 (Mean): 1,343,339.60
표준편차 (SD): 505.0 (표본 기준)



변수관리,



모델 비교

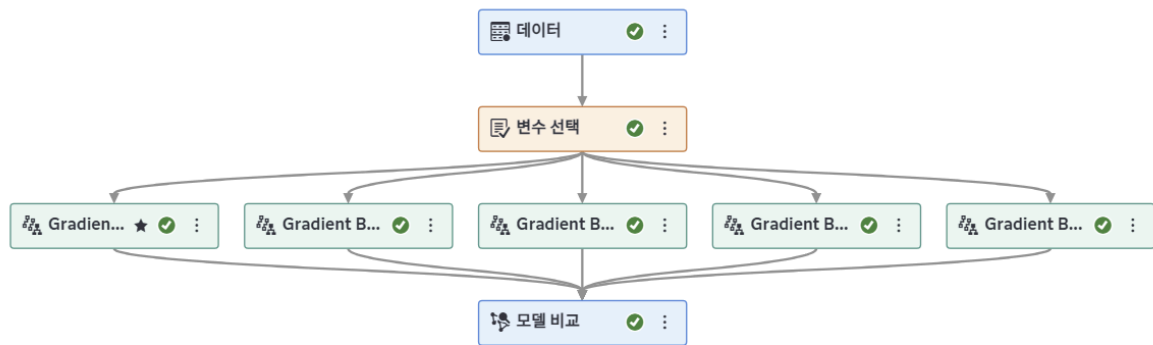
챔피언	이름	알고리즘 이름	제곱근평균제...	평균제곱오차	데C
★	Gradient Boosting_2_leaves (1)	그래디언트 부스팅	1,342,695.4175	1,802,830,984,090	VA
	Gradient Boosting_2_leaves (4)	그래디언트 부스팅	1,342,753.0729	1,802,985,814,840	VA
	Gradient Boosting_2_leaves (3)	그래디언트 부스팅	1,343,147.2555	1,804,044,550,060	VA
	Gradient Boosting_2_leaves	그래디언트 부스팅	1,343,192.6750	1,804,166,562,240	VA
	Gradient Boosting_2_leaves (2)	그래디언트 부스팅	1,343,335.6165	1,804,550,578,480	VA

평균 (Mean): 1,343,024.80

양상블 1,342,560.7548

표준편차 (SD): 283.9 (표본 기준)

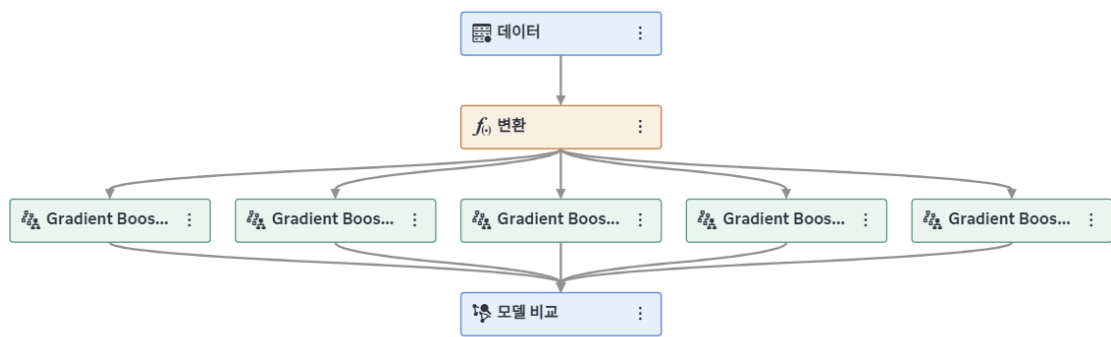
변수선택,



챔피언	이름	알고리즘 이름	제공군평균제...	평균제곱오차	데이터 역할
★	Gradient Boosting_2_leaves	그레디언트 부스팅	1,347,418.9778	1,815,537,901,840	VALIDATE
	Gradient Boosting_2_leaves (4)	그레디언트 부스팅	1,347,510.3715	1,815,784,201,210	VALIDATE
	Gradient Boosting_2_leaves (3)	그레디언트 부스팅	1,348,314.2385	1,817,951,285,800	VALIDATE
	Gradient Boosting_2_leaves (1)	그레디언트 부스팅	1,347,977.8376	1,817,044,250,620	VALIDATE
	Gradient Boosting_2_leaves (2)	그레디언트 부스팅	1,347,566.6896	1,815,935,982,920	VALIDATE

평균 (Mean): 1,347,757.62
표준편차 (SD): 377.7 (표본 기준)

변환,



모델 비교

챔피언	이름	알고리즘 이름	제곱근평균제곱오차	
★	Gradient Boosting_2_leaves (1)	그래디언트 부스팅	1,342,695.4175	1,802
	Gradient Boosting_2_leaves (4)	그래디언트 부스팅	1,342,753.0729	1,802
	Gradient Boosting_2_leaves (3)	그래디언트 부스팅	1,343,147.2555	1,804
	Gradient Boosting_2_leaves (2)	그래디언트 부스팅	1,343,335.6165	1,804
	Gradient Boosting_2_leaves	그래디언트 부스팅	1,343,192.6750	1,804

- 평균 (Mean): 1,343,024.81
- 표준편차 (SD): 283.9 (표본 기준)

피쳐머신,

모델 비교

챔피언	이름	알고리즘 이름	제공근평균제공오차	평균제공오차
★	Gradient Boosting_2_leaves (4)	그래디언트 부스팅	1,343,778.4237	1,805,740,451,870
	Gradient Boosting_2_leaves (3)	그래디언트 부스팅	1,344,360.6410	1,807,305,533,010
	Gradient Boosting_2_leaves (2)	그래디언트 부스팅	1,345,802.3079	1,811,183,851,850
	Gradient Boosting_2_leaves (1)	그래디언트 부스팅	1,344,464.8735	1,807,585,796,000
	Gradient Boosting_2_leaves	그래디언트 부스팅	1,344,377.3911	1,807,350,569,680

폐기

파이썬 연동

파이썬연동 (churn)

1. 접근법

기타 - 오픈 소스 코드 - 언어: python - 코드편집기 열기 - 학습창 (모델피팅+파생변수 생성+단순 코드) & 스코어링창 (예측값활용) - 코드 입력 - 저장 후 나가기 - 결과

1.1 유저인터페이스와 세팅

Open Source Code Editor User Interface

The following data items are defined and available for certain expected cases in your Python or R code. For R code, the `setwd(dm_nodedir)` function call is also added so that you can specify `dm_nodedir` as the current working directory. For Python code, the training code and scoring code panes are available in the editor.

Data Item	Description
dm_class_input	A list of names that identify the categorical (nominal and ordinal) variables. The data type is a list in Python and a vector in R. This variable is available in the Training Code pane and the Scoring Code pane in Python.
dm_classtarget_intovar	A variable that identifies the classification variable for a class target. The data type is a string.
dm_classtarget_level	A list of values that identify the various levels in the target variable. The data type is a list in Python and a vector in R.
dm_dec_target	A variable that identifies the name of the target. The data type is string.
dm_exposure	A variable that identifies the name of the exposure variable. The data type is string. This variable is available starting in 2025.08.
dm_freq	A variable that identifies the name of the data frequency variable. The data type is string.
dm_input	A list of names that identify the input (interval and categorical) variables. The data type is a list in Python and a vector in R.
dm_inputdf	A data frame that contains the sampled input data. This is available only when the Generate data frame property is selected. The data type is a pandas DataFrame in Python or a data frame where categorical variables are encoded as factors in R.
dm_interval_input	A list of names that identify the interval variables. The data type is a list in Python and a vector in R. This variable is available in the Training Code pane and the Scoring Code pane in Python.
dm_key	The variable that identifies the name of the key. The data type is string. This variable is available only in Python.
dm_model	An object that identifies the model. The usage of this object is optional but recommended for future use.
dm_model_formula	An object that identifies the model formula in R of the form <code>input_var1 + input_var2</code> . The data type is formula object. This object is available only in R.
dm_nodedir	A variable that identifies the node's working directory. That is, where the local files are created. The data type is string.
dm_offset	A variable that identifies the name of the offset variable. The data type is string. This variable is available starting in 2025.08.
dm_partition_train_val	A variable that identifies the value that identifies the training partition. The data type is string or integer.
dm_partitionvar	A variable that identifies the name of the data partition. The data type is string.
dm_pkname	A variable that identifies the name of the model file (pickle file in Python). This variable is available only in Python.
dm_pkpath	A variable that identifies the directory path of the model file (pickle file in Python). This variable is available only in Python (in the Training Code and Scoring Code panes).
dm_predictionvar	A list of names that identify the prediction variables. The data type is a list in Python and a vector in R.
dm_scoreddf	A data frame that you must create when the Generate data frame property is selected. The contents of this data frame depend on whether the property Use output data in child nodes is selected. When this property is not selected, and the node is executed as a Supervised Learning node, <code>dm_scoreddf</code> is expected to contain model predictions. The number of model predictions contained in <code>dm_scoreddf</code> should match the number of rows in the input data, <code>dm_inputdf</code> . This is because the input data and the model predictions are merged using the physical order of observations. When the Use output data in child nodes property is selected, <code>dm_scoreddf</code> is expected to contain output data that is used in child nodes. The data type of <code>dm_scoreddf</code> is "Pandas DataFrame" in Python and "data frame" in R.
dm_scoreinputs_path	A variable that identifies the directory path of the score inputs file in CSV format. This variable is available only in Python.
dm_scoreoutputs_path	A variable that identifies the directory path of the score outputs file in CSV format. This variable is available only in Python.
dm_traindf	A data frame that contains the sampled training data. This is available only when the Generate data frame property is selected. The data type is a pandas DataFrame in Python or a data frame where categorical variables are encoded as factors in R.
dm_weight	A variable that identifies the name of the data weight variable. The data type is string.
node_data.csv	A file in comma-separated value format that identifies the sampled input data in the <code>dm_nodedir</code> directory.
node_scored.csv	A file in comma-separated value format that you must create with model predictions when you execute the node in the Supervised Learning group and model assessment is performed. This file must be created when Generate data frame is not selected and must contain the same number of rows as <code>node_data.csv</code> .

🔥 표본 추출유형: / 관측값 백분율 - 100% 로 설정

가져온 `train_final` 데이터 중 `python` 환경으로 가져올 데이터 표본 추출 방법 및 비율 선택 옵션

1.2 데이터 출력 및 전처리

출력 코드

```
df = dm_inputdf
print(df.shape)
print(df.head())
print(df.columns.tolist())
```

결과

```
[5 rows x 40 columns]
['target_churn', 'customer_id', 'age', 'gender', 'region_code', 'is_married',
 'prefer_category', 'income_group', 'tenure_days', 'credit_score',
 'num_active_cards', 'txn_cnt_1m', 'txn_amt_sum_1m', 'recent_amt_share_3m',
 'recent_amt_share_1m', 'zero_month_count', 'amt_month_slope', 'target_ltv',
 'total_deposit_balance', 'total_loan_balance', 'card_cash_service_amt',
 'card_loan_amt', 'fin_overdue_days', 'fin_asset_trend_score',
```

```
    'loan_to_deposit', 'overdue_flag', 'txn_cnt_6m', 'txn_amt_sum_6m',
    'txn_amt_mean_6m', 'txn_amt_std_6m', 'active_months_6m',
    'days_since_last_txn', 'online_ratio_6m', 'installment_ratio_6m',
    'category_nunique_6m', 'txn_cnt_3m', 'txn_amt_sum_3m', 'net_balance',
    '_dmIndex_', '_PartInd_']
'_dmIndex_': 인덱스
'_PartInd_': train/valid (1/0 (확인완료)) 표시
```

전처리코드

```
import pandas as pd
import numpy as np
import pandas as pd
import numpy as np

df = dm_inputdf

print("==== df shape =====")
print(df.shape)

print("==== df head =====")
print(df.head())

print("==== columns =====")
print(df.columns.tolist())

print("_PartInd_ distribution:")
print(df["_PartInd_"].value_counts(dropna=False))

train_df = df[df["_PartInd_"].astype(str) == "1"]
valid_df = df[df["_PartInd_"].astype(str) == "0"]

print("train_df shape:", train_df.shape)
print("valid_df shape:", valid_df.shape)

cat_cols = ["gender", "region_code", "prefer_category", "income_group",
            "is_married", "overdue_flag"]

exclude_cols = [
    "target_churn",
    "target_ltv",
    "customer_id",
    "_dmIndex_",
    "_PartInd_"
]

def preprocess_data(input_df, train_columns=None):
    X = input_df.drop(columns=exclude_cols, errors="ignore")
```

```

y = input_df["target_churn"].astype(int) if "target_churn" in
input_df.columns else None

existing_cat_cols = [col for col in cat_cols if col in X.columns]

X_dummy = pd.get_dummies(X, columns=existing_cat_cols, drop_first=False)

if train_columns is not None:
    X_dummy = X_dummy.reindex(columns=train_columns, fill_value=0)

X_dummy = X_dummy.fillna(0)

return X_dummy, y

X_train, y_train = preprocess_data(train_df)
train_cols = X_train.columns

X_valid, y_valid = preprocess_data(valid_df, train_columns=train_cols)

print("X_train shape:", X_train.shape)
print("X_valid shape:", X_valid.shape)

print("y_train distribution:")
print(y_train.value_counts(dropna=False))

print("y_valid distribution:")
print(y_valid.value_counts(dropna=False))

print("Final X columns:")
print(X_train.columns.tolist())

# SAS 노드 완료용 임시 출력
dm_scoreddf = df.copy()
dm_scoreddf["P_target_churn0"] = 1.0
dm_scoreddf["P_target_churn1"] = 0.0
dm_scoreddf["I_target_churn"] = "0"

```

- train/valid 분리

이미 SAS Viya에서 70:30으로 나뉘기 때문에, Python에서 다시 `train_test_split()`을 하면 안 된다고 했지. 대신 Python에서는 SAS Viya가 붙여준 `_PartInd_`를 보고 이렇게 나눠야 해.

- One-hot encoding

`drop_first = T/F문제`
로지/선형 - T (다중공선성 피하기 위해)

2. TO DO

1. xgboost, cat boost 돌리기
2. sas viya환경에서 파생변수 만든 후 정상적으로 모델 돌아가는지 확인

2.1.1. Cat Boost 사용 불가

현재 SAS Viya Open Source Code 노드에서 Python 연동을 확인했습니다. 기본 환경변수인 `DM_PYPATH`에서는 `pandas`, `numpy`, `sklearn`은 사용 가능하지만 `catboost` 패키지가 설치되어 있지 않아 CatBoost 모델 실행이 불가능합니다. `DM_PYPATH2~5`도 확인했으나 해당 환경에서는 코드 실행 자체가 정상적으로 되지 않았습니다. 따라서 CatBoost를 사용하려면 Viya 관리자에게 `DM_PYPATH` 환경에 `catboost` 설치를 요청하거나, CatBoost가 설치된 Python 실행 경로를 별도 환경변수로 등록해야 합니다.

2.1.2. 파이썬 XG Boost

```
import pandas as pd
import numpy as np
import random

from xgboost import XGBClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.metrics import roc_auc_score, accuracy_score, confusion_matrix,
classification_report
```

```
SEED = 42
```

```
random.seed(SEED)
np.random.seed(SEED)
```

```
# 1. 데이터 준비
```

```
df = dm_inputdf
```

```
print("df shape:", df.shape)
print(df.head())
print("_PartInd_ distribution:")
print(df["_PartInd_"].value_counts(dropna=False))
```

```
# SAS 파티션 변수가 숫자/문자 어느 쪽이어도 작동하게 처리
train_df = df[df["_PartInd_"].astype(str) == "1"]
valid_df = df[df["_PartInd_"].astype(str) == "0"]
```

```
print("train_df shape:", train_df.shape)
```

```

print("valid_df shape:", valid_df.shape)

cat_cols = ["gender", "region_code", "prefer_category", "income_group",
"is_married", "overdue_flag"]

exclude_cols = [
    "target_churn",
    "target_ltv",
    "customer_id",
    "_dmIndex_",
    "_PartInd_"
]

# 2. 전처리 함수
def preprocess_data(input_df, train_columns=None):
    X = input_df.drop(columns=exclude_cols, errors="ignore")
    y = input_df["target_churn"].astype(int) if "target_churn" in
input_df.columns else None

    existing_cat_cols = [col for col in cat_cols if col in X.columns]

    X_dummy = pd.get_dummies(X, columns=existing_cat_cols, drop_first=False)

    if train_columns is not None:
        X_dummy = X_dummy.reindex(columns=train_columns, fill_value=0)

    X_dummy = X_dummy.fillna(0)

    return X_dummy, y

X_train, y_train = preprocess_data(train_df)
train_cols = X_train.columns

X_valid, y_valid = preprocess_data(valid_df, train_columns=train_cols)

print("X_train shape:", X_train.shape)
print("X_valid shape:", X_valid.shape)
print("y_train distribution:")
print(y_train.value_counts(dropna=False))
print("y_valid distribution:")
print(y_valid.value_counts(dropna=False))

# 3. XGBoost 모델 설정
xgb_model = XGBClassifier(
    n_estimators=300,
    max_depth=3,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,

```

```

        objective="binary:logistic",
        eval_metric="auc",
        random_state=SEED
    )

# 4. 5-Fold Stratified Cross Validation 수행
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
cv_auc_scores = cross_val_score(xgb_model, X_train, y_train, cv=cv,
                                scoring="roc_auc")

# 5. 최종 모델 학습 및 검증 데이터 평가
xgb_model.fit(X_train, y_train)

valid_prob = xgb_model.predict_proba(X_valid)[:, 1]
valid_pred = (valid_prob >= 0.5).astype(int)

valid_auc = roc_auc_score(y_valid, valid_prob)
valid_acc = accuracy_score(y_valid, valid_pred)

print("==== XGBoost 5-Fold CV Result ====")
print("CV AUC scores:", cv_auc_scores)
print("Mean CV AUC:", round(np.mean(cv_auc_scores), 4))
print("Std CV AUC:", round(np.std(cv_auc_scores), 4))

print("\n==== Final Validation Result ====")
print("Validation AUC-ROC:", round(valid_auc, 4))
print("Validation Accuracy:", round(valid_acc, 4))

print("\nConfusion Matrix:")
print(confusion_matrix(y_valid, valid_pred))

print("\nClassification Report:")
print(classification_report(y_valid, valid_pred, zero_division=0))

result_df = pd.DataFrame({
    "model": ["XGBoost"],
    "CV_Mean_AUC": [round(np.mean(cv_auc_scores), 5)],
    "CV_Std_AUC": [round(np.std(cv_auc_scores), 6)],
    "Validation_AUC_ROC": [round(valid_auc, 6)],
    "Validation_Accuracy": [round(valid_acc, 6)]
})

print("\n==== Model Performance Summary ====")
print(result_df.to_string(index=False))

# 6. SAS 결과 데이터 생성

```

```
X_all, _ = preprocess_data(df, train_columns=train_cols)
all_probs = xgb_model.predict_proba(X_all)[:, 1]

dm_scoreddf = df.copy()
dm_scoreddf["P_target_churn1"] = all_probs
dm_scoreddf["P_target_churn0"] = 1.0 - all_probs
dm_scoreddf["I_target_churn"] = np.where(all_probs >= 0.5, "1", "0")
```

해설.

관측값 백분율을 100%로 설정하여 전체 데이터를 Python Open Source Code 노드로 가져온 후 XGBoost 모델을 학습하였다. SAS Viya에서 사전에 분할한 학습 데이터 내부에서 5-Fold Stratified Cross Validation을 수행한 결과, 평균 AUC-ROC는 0.7932, 표준편차는 0.0092로 나타났다. 이후 전체 학습 데이터로 최종 모델을 학습하고 검증 데이터에서 평가한 결과, Validation AUC-ROC는 0.7967, Accuracy는 0.9132로 나타났다. churn 데이터는 이탈 고객 비율이 낮은 불균형 데이터이므로, 최종 모델 성능은 Accuracy보다 AUC-ROC를 중심으로 판단하였다. 따라서 XGBoost 모델은 검증 데이터 기준으로 이탈 고객과 비이탈 고객을 어느 정도 양호하게 구분하는 성능을 보였다고 해석할 수 있다.

문제.

1. 이전 모델도 cv 진행해야 함 -> 모델 노드와 섞을 수 없음,, 문제..
2. 표본추출의 문제 있음 -> 표본 추출 비율 100% 으로 해결!

Q. 스코어링 코드

1:29:14~

헤리 컴퓨터로 작업

Var.Sel. 2 + XGB

cus

churn-final_first

1. 데이터 정제

헤리가 해준 기준으로 추가 및 삭제된 내용 정리

- **target_churn** : Target/Binary
- **target_ltv** : Rejected
- **target_ltv_log** : Rejected
- **customer_id** : ID/Nominal
- **net_balance** : input/interval
- **active_months_6m** : input/ interval
- **category_nunique_6m** : input/ interval
- **fin_overdue_days** : input/ interval
- **num_active_cards** : input/ interval
- **region_code** : input/nominal

2. 알고리즘 탐색

- 2.1 챔피언 모델 선정 - 그래디언트부스팅

모델 비교

챔피언	이름	알고리즘 ...	KS(You...	정확도	평균제공...	ROC 아...	누적향상도	누적반응...	절사값	데이터 역할
★	그래디언트 부스팅	그래디언트 부스팅	0.4999	0.9144	0.0705	0.7980	4.0641	40.6408	0.5000	VALIDATE
	포리스트	포리스트	0.4937	0.9126	0.0716	0.7913	3.9236	39.2355	0.5000	VALIDATE
	의사결정트리	의사결정트리	0.4495	0.9136	0.0729	0.7467	3.6711	36.7110	0.5000	VALIDATE
	로지스틱 회귀	로지스틱 회귀	0.3561	0.9027	0.0802	0.7381	3.2940	32.9399	0.5000	VALIDATE

○

변수명	의미	역할	레벨
active_months_6m	최근 6개월 활동 월 수	Input	Interval
age	연령	Input	Interval
amt_month_slope	월별 거래금액 변화 추세	Input	Interval
card_cash_service_amt_0	카드 현금서비스 금액 처리 변수	Input	Interval
card_cash_service_amt_log	카드 현금서비스 금액 로그 변환	Input	Interval
card_loan_amt_log	카드론 금액 로그 변환	Input	Interval
category_nunique_6m	최근 6개월 사용 카테고리 종류 수	Input	Interval
credit_score	신용점수	Input	Interval
customer_id	고객 식별자	ID	Nominal
days_since_last_txn	마지막 거래 이후 경과일	Input	Interval

fin_asset_trend_score	금융자산 추세 점수	Input	Interval
fin_overdue_days	금융 연체일 수	Input	Interval
gender	성별	Input	Binary
income_group	소득 구간	Input	Nominal
installment_ratio_6m	최근 6개월 할부 거래 비율	Input	Interval
is_married	결혼 여부	Input	Binary
loan_to_deposit_cut	예금 대비 대출 비율 처리 변수	Input	Interval
net_balance	순잔고	Input	Interval
num_active_cards	활성 카드 수	Input	Interval
online_ratio_6m	최근 6개월 온라인 거래 비율	Input	Interval
overdue_flag	연체 여부	Input	Binary

prefer_category	선호 소비 카테고리	Input	Nominal
recent_amt_share_1m	최근 1개월 거래금액 비중	Input	Interval
recent_amt_share_3m	최근 3개월 거래금액 비중	Input	Interval
region_code	거주/지역 코드	Input	Nominal
target_churn	고객 이탈 여부	Target	Binary
target_ltv	고객 생애가치 원 변수	Rejected	Interval
target_ltv_log	고객 생애가치 로그 변환 변수	Rejected	Interval
tenure_days	고객 유지 기간	Input	Interval
total_deposit_balance_log	총 예금잔액 로그 변환	Input	Interval
total_loan_balance_log	총 대출잔액 로그 변환	Input	Interval
txn_amt_mean_6m	최근 6개월 평균 거래금액	Input	Interval

txn_amt_std_6m	최근 6개월 거래금액 표준편차	Input	Interval
txn_amt_sum_1m	최근 1개월 총 거래금액	Input	Interval
txn_amt_sum_3m	최근 3개월 총 거래금액	Input	Interval
txn_amt_sum_6m	최근 6개월 총 거래금액	Input	Interval
txn_cnt_1m	최근 1개월 거래건수	Input	Interval
txn_cnt_3m	최근 3개월 거래건수	Input	Interval
txn_cnt_6m	최근 6개월 거래건수	Input	Interval

- 코드

```
import pandas as pd

import numpy as np

from xgboost import XGBClassifier

from sklearn.model_selection import StratifiedKFold, cross_val_score

from sklearn.metrics import roc_auc_score, accuracy_score, confusion_matrix,
classification_report
```

```
# 1. 데이터 준비
```

```
df = dm_inputdf
```

```
# SAS 파티션 변수가 대문자일 경우를 대비해 소문자로 변경하거나 직접 참조
```

```
train_df = df[df["_PartInd_"] == 1]
```

```
valid_df = df[df["_PartInd_"] == 0]
```

```
#범주형자료 컬럼
```

```
cat_cols = ["gender", "income_group", "is_married", "overdue_flag",  
            "prefer_category", "region_code"]
```

```
#제외 컬럼
```

```
exclude_cols = ["target_churn", "target_ltv", "customer_id", "_dmIndex_",  
                "_PartInd_"]
```

```
# 2. 전처리 함수
```

```
def preprocess_data(input_df, train_columns=None):
```

```
    X = input_df.drop(columns=exclude_cols, errors="ignore")
```

```
    y = input_df["target_churn"] if "target_churn" in input_df.columns else None
```

```
    X_dummy = pd.get_dummies(X, columns=cat_cols, drop_first=False)
```

```
    if train_columns is not None:
```

```
        X_dummy = X_dummy.reindex(columns=train_columns, fill_value=0)
```

```
    X_dummy = X_dummy.fillna(0)
```

```
    return X_dummy, y
```

```
X_train, y_train = preprocess_data(train_df)
```

```
train_cols = X_train.columns
```

```
X_valid, y_valid = preprocess_data(valid_df, train_columns=train_cols)
```

```
# 3. XGBoost 모델 설정 (챔피언 설정 유지)
```

```
xgb_model = XGBClassifier(  
  
    n_estimators=300,  
  
    max_depth=3,  
  
    learning_rate=0.05,  
  
    subsample=0.8,  
  
    colsample_bytree=0.8,  
  
    objective="binary:logistic",  
  
    eval_metric="auc",  
  
    random_state=42  
  
)
```

```
# 4. 5-Fold Stratified Cross Validation 수행
```

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)  
  
cv_auc_scores = cross_val_score(xgb_model, X_train, y_train, cv=cv,  
                                scoring='roc_auc')
```

```
# 5. 최종 모델 학습 및 검증 데이터 평가
```

```
xgb_model.fit(X_train, y_train)  
  
  
  
valid_prob = xgb_model.predict_proba(X_valid)[:, 1]  
  
valid_pred = (valid_prob >= 0.5).astype(int)  
  
  
  
valid_auc = roc_auc_score(y_valid, valid_prob)  
  
valid_acc = accuracy_score(y_valid, valid_pred)
```

```

# =====

# [최종 요약 리포트 양식] - 친구 코드 포맷과 일치

# =====

print("==== XGBoost 5-Fold CV Result ====")

print("CV AUC scores:", cv_auc_scores)

print("Mean CV AUC:", round(np.mean(cv_auc_scores), 4))

print("Std CV AUC:", round(np.std(cv_auc_scores), 4))


print("\n==== Final Validation Result ====")

print("Validation AUC-ROC:", round(valid_auc, 4))

print("Validation Accuracy:", round(valid_acc, 4))


print("\nConfusion Matrix:")

print(confusion_matrix(y_valid, valid_pred))


print("\nClassification Report:")

print(classification_report(y_valid, valid_pred, zero_division=0))


# 결과 요약표 생성

result_df = pd.DataFrame({

    "model": ["XGBoost"],

    "CV_Mean_AUC": [round(np.mean(cv_auc_scores), 5)],

    "CV_Std_AUC": [round(np.std(cv_auc_scores), 6)],

    "Validation_AUC_ROC": [round(valid_auc, 6)],

    "Validation_Accuracy": [round(valid_acc, 6)]

})

```

```
print("\n==== Model Performance Summary =====")

print(result_df.to_string(index=False))


# 6. SAS 결과 데이터 생성 (기존 로직 유지)

X_all, _ = preprocess_data(df, train_columns=train_cols)

all_probs = xgb_model.predict_proba(X_all)[:, 1]


dm_scoreddf = df.copy()

dm_scoreddf["P_target_churn1"] = all_probs

dm_scoreddf["P_target_churn0"] = 1.0 - all_probs

dm_scoreddf["I_target_churn"] = np.where(all_probs >= 0.5, "1", "0")
```

ltv 모델

Itv_log 모델

- **capping 작업**

: 데이터 전처리 단계에서 '**상한선 설정**' 또는 '뚜껑 씌우기'라고 할 수 있습니다. 데이터 분석이나 머신러닝을 할 때, 비정상적으로 너무 크거나 작은 값(**이상치, Outlier**)이 전체 모델의 흐름을 방해하지 못하도록 특정 범위 안에 **가두는 작업**입니다.

따라서 지금 RMSE가 높고 R2가 음수인 이유는, 로그 복원 과정에서 저 **이상치 값이 기하급수적으로 튀어 올랐기 때문에** capping 작업이 필요합니다.

Q. log를 취해서 튀는 변수를 우리가 제압해준 거잖아 근데 왜 또 capping을 하는 거야?

A. 결론부터 말하면, 로그는 '**힘**'을 뺀 '**이상치**'라는 사실 자체를 지우지는 못하기 때문입니다.

- (1) 로그를 취해도 '서열'은 그대로 남습니다 (상대적 격차)
- (2) 로그 복원(**exp**) 시 오차가 기하급수적으로 폭발합니다 (main reason)
- (3) 입력 변수(X)의 이상치는 로그가 안 되어 있을 수 있습니다

- 데이터 정제

변수명	역할	레벨
active_months_6m	Input	Interval
age	Input	Interval
amt_month_slope	Input	Interval
card_cash_service_amt_0	Input	Interval
card_cash_service_amt_log	Input	Interval
card_loan_amt_log	Input	Interval
category_nunique_6m	Input	Interval
credit_score	Input	Interval
customer_id	ID	Nominal
days_since_last_txn	Input	Interval
fin_asset_trend_score	Input	Interval
fin_overdue_days	Input	Interval
gender	Input	Nominal
income_group	Input	Nominal
installment_ratio_6m	Input	Interval
is_married	Input	Nominal
loan_to_deposit_cut	Input	Interval
net_balance	Input	Interval

변수명	역할	레벨
num_active_cards	Input	Interval
online_ratio_6m	Input	Interval
prefer_category	Input	Nominal
recent_amt_share_1m	Input	Interval
recent_amt_share_3m	Input	Interval
region_code	Input	Nominal
target_churn	Rejected	Binary
target_ltv	Rejected	Interval
target_ltv_log	Target	Interval
tenure_days	Input	Interval
total_deposit_balance_log	Input	Interval
total_loan_balance_log	Input	Interval
txn_amt_mean_6m	Input	Interval
txn_amt_std_6m	Input	Interval
txn_amt_sum_1m	Input	Interval
txn_amt_sum_3m	Input	Interval
txn_amt_sum_6m	Input	Interval
txn_cnt_1m	Input	Interval
txn_cnt_3m	Input	Interval
txn_cnt_6m	Input	Interval

- 사용 코드

```
import subprocess
import sys
import os
```

```
# 1. 패키지를 설치할 임시 경로 지정 (SAS Viya 서버에서 쓰기 권한이 있는 폴더)
install_path = "/tmp/custom_python_libs"
if not os.path.exists(install_path):
    os.makedirs(install_path)
```

```
# 2. 파이썬 코드 내부에서 pip install 강제 실행
```

```
try:
    print("CatBoost 설치를 시도합니다...")
    # --target 옵션을 주어 지정한 폴더에만 설치되도록 격리합니다.
    subprocess.check_call(
        [sys.executable, "-m", "pip", "install", "catboost", "--target", install_path]
    )
    print("설치 완료!")
except subprocess.CalledProcessError as e:
    print(f"설치 중 시스템 오류 발생: {e}")
```

```
# 3. 설치된 폴더를 파이썬 라이브러리 경로(sys.path)의 가장 앞에 추가
if install_path not in sys.path:
    sys.path.insert(0, install_path)
```

```
# 4. 이제 기존 코드 실행 (import)
```

```
try:
    import catboost
    from catboost import CatBoostRegressor
    print(f"CatBoost 로드 성공! 버전: {catboost.__version__}")
except ModuleNotFoundError as e:
    print(f"로드 실패: {e}")
```

```
# --- 아래부터는 원래 작성하시려던 CatBoost 모델링 코드를 이어가시면 됩니다 ---
```

```
import pandas as pd
import numpy as np
import random
import os
```

```

from xgboost import XGBRegressor
from catboost import CatBoostRegressor
from sklearn.linear_model import ElasticNetCV
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import KFold

def seed_everything(seed=42):
    random.seed(seed)
    os.environ["PYTHONHASHSEED"] = str(seed)
    np.random.seed(seed)

seed_everything(42)

df = dm_inputdf.copy()

train_mask = (df["_PartInd_"] == 1)

drop_cols = [
    "target_ltv",
    "target_ltv_log",
    "target_churn",
    "customer_id",
    "_dmlIndex_",
    "_PartInd_"
]

X_all = df.drop(columns=drop_cols, errors="ignore")
X_all = pd.get_dummies(X_all, drop_first=True)
X_all = X_all.replace([np.inf, -np.inf], np.nan).fillna(0)

y_full = df.loc[train_mask, "target_ltv"].values

# 상위 0.2% 제거
limit = np.percentile(y_full, 99.8)
clean_idx = df[train_mask & (df["target_ltv"] <= limit)].index

X_train_clean = X_all.loc[clean_idx].reset_index(drop=True)
y_train_clean = df.loc[clean_idx, "target_ltv"].values

kf = KFold(n_splits=5, shuffle=True, random_state=42)

oof_preds = np.zeros((len(X_train_clean), 2))
all_preds = np.zeros((len(X_all), 2))

```

```
print("🌟 LTV original-scale stacking model start")
```

```
for fold, (tr_idx, val_idx) in enumerate(kf.split(X_train_clean)):
```

```
    X_tr = X_train_clean.iloc[tr_idx]
```

```
    X_val = X_train_clean.iloc[val_idx]
```

```
    y_tr = y_train_clean[tr_idx]
```

```
    y_val = y_train_clean[val_idx]
```

```
    xgb = XGBRegressor(
```

```
        n_estimators=1200,
```

```
        learning_rate=0.01,
```

```
        max_depth=6,
```

```
        subsample=0.8,
```

```
        colsample_bytree=0.8,
```

```
        reg_lambda=100,
```

```
        random_state=42,
```

```
        objective="reg:squarederror",
```

```
        n_jobs=-1
```

```
    )
```

```
    cat = CatBoostRegressor(
```

```
        iterations=1200,
```

```
        learning_rate=0.01,
```

```
        depth=7,
```

```
        l2_leaf_reg=30,
```

```
        loss_function="RMSE",
```

```
        random_seed=42,
```

```
        verbose=0
```

```
    )
```

```
    xgb.fit(X_tr, y_tr)
```

```
    cat.fit(X_tr, y_tr)
```

```
    oof_preds[val_idx, 0] = xgb.predict(X_val)
```

```
    oof_preds[val_idx, 1] = cat.predict(X_val)
```

```
    all_preds[:, 0] += xgb.predict(X_all) / 5
```

```
    all_preds[:, 1] += cat.predict(X_all) / 5
```

```
    fold_pred = (oof_preds[val_idx, 0] + oof_preds[val_idx, 1]) / 2
```

```
    fold_rmse = np.sqrt(mean_squared_error(y_val, fold_pred))
```

```
    print(f"✅ Fold {fold+1} 완료 / Fold RMSE: {fold_rmse:,.2f}")
```

```
meta_model = ElasticNetCV(cv=5, random_state=42)
```

```
meta_model.fit(oof_preds, y_train_clean)

oof_final = meta_model.predict(oof_preds)
oof_final = np.clip(oof_final, 0, None)

oof_rmse = np.sqrt(mean_squared_error(y_train_clean, oof_final))
print(f"\n 🚩 Clean OOF RMSE: {oof_rmse:,.2f} 원")

final_preds = meta_model.predict(all_preds)
final_preds = np.clip(final_preds, 0, None)

dm_scoreddf = df.copy()
dm_scoreddf["P_target_ltv"] = final_preds
```

